LECTURE NOTES · OPEN MODELS, 2026

Post-Training, Visually

Companion notes to the slides

How a base model becomes an assistant, as practised on open-weight models: supervised fine-tuning, preference optimization, reinforcement learning with verifiable rewards, reasoning, long context, safety and tool use, and the recipes that combine them. One section per slide, with every term defined and the intuition spelled out, and a reading list for each part.

For advanced undergraduates and beginning graduate students

October 2026

These notes follow the deck *Post-Training, Visually* (32 slides) slide by slide. Numbers quoted from a paper were checked against that paper; curves the slides label "illustrative" are sketches of a mechanism, not data; hyperparameter ranges are indicative. Claims marked † come from release notes, model cards, blog posts or commentary rather than papers.

Contents

How to use these notes **3**

1 Overview (slides 1–3) **3**
1.1 Slide 1: Title: Post-Training, Visually . . . 3
1.2 Slide 2: From base model to assistant . . . 4
1.3 Slide 3: Three kinds of signal, three objectives . . . 5

2 Supervised fine-tuning (slides 4–6) **8**
2.1 Slide 4: SFT I: the objective is pretraining loss on the answer only . . . 8
2.2 Slide 5: SFT II: where the demonstrations come from . . . 9
2.3 Slide 6: SFT III: rejection sampling, the bridge to RL . . . 11

3 Preference optimization (slides 7–11) **13**
3.1 Slide 7: RLHF: the original three steps . . . 13
3.2 Slide 8: Reward models, and why you cannot optimize them forever . . . 14
3.3 Slide 9: PPO for language models . . . 16
3.4 Slide 10: DPO: skip the reward model . . . 17
3.5 Slide 11: Choosing a preference method . . . 19

4 Reinforcement learning (slides 12–14) **21**
4.1 Slide 12: GRPO: group-relative advantages, no value model . . . 21
4.2 Slide 13: RL with verifiable rewards (RLVR) . . . 22
4.3 Slide 14: The RL loop at scale, and what breaks . . . 24

5 Reasoning (slides 15–18) **26**
5.1 Slide 15: Thinking tokens: spend compute at inference . . . 26
5.2 Slide 16: DeepSeek-R1: RL makes the model think longer . . . 27
5.3 Slide 17: Distillation, thinking budgets, and one model for both modes . . . 28
5.4 Slide 18: What reasoning RL does, and does not, add . . . 30

6 Long context (slides 19–21) **32**
6.1 Slide 19: Context: why a long window is its own stage . . . 32
6.2 Slide 20: Stretching RoPE: interpolation, base change, YaRN . . . 33
6.3 Slide 21: The long-context protocol: stages, data, and checks . . . 34

7 Safety (slides 22–24) **37**
7.1 Slide 22: Safety alignment: targets and data . . . 37
7.2 Slide 23: Measuring safety: red teams, jailbreaks and guards . . . 38
7.3 Slide 24: Open weights: fine-tuning undoes alignment . . . 40

8 Tool use (slides 25–26) **42**
8.1 Slide 25: Tool use I: function calling is a format, taught by SFT . . . 42
8.2 Slide 26: Tool use II: agentic RL over many turns . . . 43

9 Recipes and evaluation (slides 27–30) 45
9.1 Slide 27: Open recipes, side by side . . . 45
9.2 Slide 28: On-policy distillation: dense signal, student's samples . . . 46
9.3 Slide 29: Evaluating each stage, and the regressions to watch . . . 47
9.4 Slide 30: A practical plan for a small team . . . 49

10 Wrap-up (slides 31–32) 51
10.1 Slide 31: Summary: what each stage adds, and what it costs . . . 51
10.2 Slide 32: Sources . . . 51

A Notation 53

B Formulas at a glance 54

C Exercises 55

D Reading list 57

Index of terms 64

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the equations written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Conventions. π_θ is the policy being trained and π_{ref} the frozen reference (usually the SFT model); x is a prompt and y a response, y_t its t-th token; $r(x, y)$ is a scalar reward, from a learned reward model r_ϕ or a verifier; $\mathrm{KL}(\pi \,\|\, \pi_{\text{ref}})$ is the Kullback–Leibler divergence. "On-policy" means the training data were sampled from the current policy; "off-policy" means they were not. Dataset sizes are quoted as the papers give them (prompts, pairs or examples, which differ).

A reading plan for a single lecture: Sections 1–3 (SFT, preference, RL with verifiable rewards) are the core and take about half the session; Section 4 (reasoning) is where the 2025–26 models live and takes a quarter; Sections 5–7 (context, safety, tools) can be sampled, each being a self-contained data problem; Sections 8–9 (recipes, evaluation, the plan) are what students should be able to do afterwards. Appendix D is the reading list, section by section, and matches the paper archive that accompanies these notes.

1 Overview (slides 1–3)

1.1 Slide 1: Title: Post-Training, Visually

How a base model becomes an assistant: supervised fine-tuning, preference optimization, reinforcement learning, reasoning, long context, safety and tool use, as practised on open-weight models.

What the slide shows A staircase of seven bars, each longer and higher than the last: base model, SFT, preference, RL, reasoning, safety, tools. An arrow beside it reads "more capable, more aligned". The picture is the lecture's claim in one shape: post-training is a sequence of stages stacked on one base model, each adding a behavior, and the stages are ordered. The order is partly logical (you cannot run RL on a model that does not yet answer in the right format) and partly historical (the stages were invented in roughly this order).

Details The deck has ten footer stages: Overview, SFT, Preference, RL, Reasoning, Context, Safety, Tools, Recipes, Wrap-up. The first four are the core machinery, in increasing order of what the training signal costs and what it can reach. Reasoning is the application of that machinery that dominated 2025 and 2026. Context, safety and tool use are three further behaviors, each of which turns out to be mostly a data problem with its own evaluations. Recipes puts the stages together as the open labs actually did, and ends with a plan a small team can run.

The scope is *open-weight* models: those whose weights are published, so that the recipe can be read from the paper or the release notes and, often, reproduced. Closed models follow the same stages, but the only evidence for what they do is what their makers say. Where the deck quotes a closed model (InstructGPT, o1, GPT-3.5 fine-tuning) it is because the paper or experiment is the origin of a technique or a result that the open models then adopted.

Two words in the title deserve a definition. *Post-training* is everything done to a model after its pretraining run: all the stages here, plus the evaluation between them. The older term *fine-tuning* is now used for one of the stages (SFT) or for the generic act of continuing training on new data; *alignment* is used both for the whole of post-training and, narrowly, for the safety part.

Terms

base model A model that has only been pretrained on next-token prediction over a large corpus. It continues text; it does not, by default, answer questions, follow instructions or stop at the end of an answer.

post-training All training after pretraining: supervised fine-tuning, preference optimization, reinforcement learning, context extension, safety training and tool-use training, in whatever order a recipe chooses. It uses a small fraction of the pretraining compute and changes behavior more than knowledge.

open-weight model A model whose trained parameters are published for download, with or without the training data and code. Llama, Qwen, DeepSeek, Mistral, OLMo and Kimi are the open-weight families the deck draws on.

alignment Making a model's behavior match what its developers and users intend. In the broad sense it is a synonym for post-training; in the narrow sense it is the safety stage, which teaches the model what to refuse.

recipe The particular sequence of post-training stages, data sources and hyperparameters a lab used for one model release. Recipes are compared side by side on slide 27.

Intuition

A base model is a very well-read person who has never been asked a question. Post-training does not make the person more learned; it teaches them to notice they are being asked something, to answer in a form the asker can use, to prefer good answers to bad ones, to check their work, to say no to some requests, and to pick up a tool when one is needed. Every stage in the staircase is one of those lessons.

1.2 Slide 2: From base model to assistant

Pretraining produces a text predictor. Post-training turns it into something that answers, reasons, refuses, and acts.

What the slide shows A chevron pipeline across the top: pretraining, mid-training, SFT, preference, RL, safety, release, each with a one-line note on what it consumes (demonstrations, comparisons, rewards, refusals and evaluations). Below, the same prompt given to two models. The base model, asked to explain why the sky is blue, continues the text as if it were in a textbook: "Explain why grass is green. Explain why… (Chapter 3 review questions)". The post-trained model answers the question. Three readings at the bottom: behavior, not knowledge; three signals; what changed by 2026.

Details The base model's continuation is the central observation. The model has seen the Rayleigh-scattering explanation many times during pretraining and could produce it; what it produces instead is the most likely continuation of a line that looks like a review question, which is another review question. Nothing about the model's knowledge is wrong. What is missing is a mapping from "a question in a prompt" to "an answer in the response", and that mapping is what post-training installs. This is why post-training is cheap relative to pretraining: it is a small change in which continuation is produced, not a large change in what the model knows. InstructGPT's SFT used 13k prompts after pretraining on hundreds of billions of tokens;

LIMA (slide 5) showed that a thousand examples teach the format.

The pipeline chevrons name the three signals that the rest of the deck is organized around. SFT consumes demonstrations: prompt and ideal response. Preference optimization consumes comparisons: two responses and a label saying which is better. RL consumes rewards: a score for a response the model itself produced. The safety chevron is listed separately, as it is in most recipes, but it uses the same three signals on a particular kind of data. Mid-training, between pretraining and SFT, is where the context window is extended and the data mix is shifted toward higher quality; it is covered in Section 5.

The third reading names what moved between the 2022 pipeline and the 2026 one. In InstructGPT RL was the final polish, applied for a few tens of thousands of prompts after SFT. In the 2025–26 open models RL is the centre of the recipe, run for thousands of steps on verifiable tasks, often in several specialist runs (math, code, agents) whose results are then merged into one model by on-policy distillation (slide 28). The staircase of slide 1 is still the right picture; the sizes of the steps changed.

Terms

pretraining Training on next-token prediction over trillions of tokens of text and code. It produces the base model and consumes almost all of the compute.

mid-training A late phase of pretraining with a changed data mix (higher quality, more code and math, long documents) and usually the context extension. The term is informal; some reports fold it into pretraining and others into post-training.

supervised fine-tuning (SFT) Continuing to train the base model on prompt–response demonstrations with the next-token loss on the response. Also called instruction tuning when the demonstrations are instructions and answers.

preference optimization Training from comparisons between responses rather than from a single correct response. RLHF with a reward model and DPO are the two main forms.

reinforcement learning (RL) Training in which the model samples responses, receives a scalar reward for each, and is updated to make high-reward responses more likely. The data are produced by the model during training rather than collected beforehand.

demonstration A prompt paired with a response that the model should learn to produce. The data item of SFT.

comparison A prompt with two (or more) responses and a judgment of which is better. The data item of preference optimization.

reward A scalar score assigned to a response, by a learned reward model, a program that checks the answer, or a judge model. The data item of RL.

assistant The post-trained model in its deployed role: it takes a conversation with system, user and assistant turns and produces the next assistant turn.

Intuition

The base model is a mirror of its training text: show it the beginning of a textbook page and it shows you the rest of the page. Post-training changes the mirror into a window: show it a question and it shows you an answer. The glass is the same; what changed is which side of it the model thinks it is on.

1.3 Slide 3: Three kinds of signal, three objectives

Every post-training method is one of these, or a combination. The data item decides the loss.

What the slide shows Three cards, one per signal. Each names the data item, the objective trained on it, and what it teaches. Demonstrations: a prompt and a good response; maximize $\log \pi(y \mid x)$ on the response tokens; teaches format and style, and is cheap to get from a stronger model. Comparisons: a prompt and two responses with a label; raise $\pi(y_w)$ relative to $\pi(y_l)$, through a Bradley–Terry reward model or DPO directly; teaches taste, and needs no correct answer. Rewards: a prompt, model samples, a verifier or reward model scoring each; policy gradient raises the probability of responses that scored above average; teaches behaviors that are in no dataset, and is the one signal that scales with compute. The footer notes that the order is also the history: SFT 2021–22, comparisons 2022–23, rewards from verifiers at scale 2025.

Details The claim that every method is one of the three is worth checking against the rest of the deck. Rejection sampling (slide 6) is rewards used to select demonstrations, then SFT. RLHF (slide 7) is comparisons used to fit a reward model, then RL against it. DPO (slide 10) is comparisons used directly. GRPO with a verifier (slides 12–13) is rewards. Constitutional AI (slide 22) is demonstrations generated by the model under critique, then comparisons labelled by the model. On-policy distillation (slide 28) is a reward per token supplied by a teacher's log-probabilities. The three objectives are the alphabet; the methods are words.

The three cards are ordered by what the signal costs and what it can reach. A demonstration must be written, by a person or a stronger model, and the student can only learn to imitate it; if no one can write a good enough response, SFT cannot teach it. A comparison only requires someone to judge which of two responses is better, which is far easier than writing the best one, and the judge need not be able to produce either; but the responses still come from somewhere, and if neither is good the comparison teaches little. A reward requires only a way to score a response the model produced itself. Nothing need be written, and the model can find a behavior no one demonstrated, which is why RL is the signal that improves with more compute: more samples means more chances to find a high-reward response, and more updates move the policy toward them. The price is that the reward must be trustworthy, since the model will find whatever the scorer rewards, including mistakes in the scorer (slide 8).

"Teaches" is also the order in which the signals are usually applied. Format must come first, so that the model produces responses that can be compared or scored at all. Taste comes next, because it is cheap and because a reward model is itself trained from comparisons. Rewards come last and, in the recipes of 2025–26, longest.

Terms

objective The quantity the optimizer maximizes or minimizes during training; the loss is the negative of it. Each signal has its own objective, written on the card.

policy The model viewed as a probability distribution over responses given a prompt, $\pi_\theta(y \mid x)$. The word comes from reinforcement learning, where the model's choice of next token is an action.

policy gradient The family of RL algorithms that update the policy's parameters in the direction that raises the log-probability of actions in proportion to how much better than expected they turned out. REINFORCE, PPO and GRPO are all policy-gradient methods.

verifier A program that decides whether a response is correct: an answer checker, a set of unit tests, a constraint checker. It supplies a reward without a learned model.

reward model (RM) A neural network trained on comparisons to output a scalar score for a response. It replaces the human judge during RL.

synthetic data Training examples generated by a model rather than written by people. Most open SFT and preference data today is synthetic, produced by a stronger model and filtered.

Bradley–Terry model A model of pairwise comparisons in which the probability that a beats b is a

logistic function of the difference of their scores. It is the statistical basis of reward-model training (slide 8).

Intuition

Three ways to teach someone to cook. Show them a finished dish and the recipe (a demonstration); they will reproduce it, and only it. Taste two of their attempts and say which was better (a comparison); they can now improve what they already cook. Let them cook, taste the result, and score it (a reward); now they can invent. The third is the only one that can produce a dish you could not have made yourself, and the only one where a broken palate on the judge's part is catastrophic.

2 Supervised fine-tuning (slides 4–6)

2.1 Slide 4: SFT I: the objective is pretraining loss on the answer only

A conversation is rendered through a chat template; the loss is taken on assistant tokens and masked everywhere else.

What the slide shows A ribbon of tokens for a short conversation: <|system|> You are helpful. <|user|> Sum 1 to 10. <|assistant|> The sum is 55. <|end|>. The first ten tokens are grey and braced "masked: no loss (context only)"; the last five, from "The" to <|end|>, are coloured and braced "trained: next-token loss". Beneath it the loss, then two notes: the loss is the pretraining loss restricted to the assistant's tokens, and the template is part of the model. A panel on the right gives hyperparameters, full fine-tuning against LoRA, and what SFT cannot do.

Details The objective is

$$\mathcal{L}_{\text{SFT}} = -\sum_{t \in \text{assistant}} \log \pi_\theta(y_t \mid x, y_{<t})$$

where the sum runs over the token positions that belong to assistant turns, and the conditioning context at each position is everything before it, including the system and user turns and any earlier assistant turns. This is exactly the pretraining loss with a mask. The mask matters for two reasons. The user's tokens are not something the model should learn to produce; training on them would teach the model to write questions, and in multi-turn data would spend most of the gradient on text the model never generates. And tool outputs (Section 8) are produced by the environment, not the model; training on them teaches the model to hallucinate tool results.

The chat template is the function that turns a structured conversation (a list of roles and messages) into the token sequence the model sees. It inserts the special role tokens and the end-of-turn token, and, for tool use, the tool schemas. The special tokens are new entries in the vocabulary, with embeddings learned during SFT. The model learns to emit <|end|> when its answer is complete, which is how the deployed model knows to stop; a base model has no such token and runs on. Because the template is learned, it is part of the model: serving a model with a different template than it was trained on degrades it, and every model family has its own.

The panel's hyperparameters are small because SFT starts from a trained model and must not destroy it. A learning rate of 10^{-5} to 2×10^{-5} is roughly a tenth of a typical pretraining peak; one to three epochs over the data; sequences packed end to end to fill the context length, with attention masked across the boundaries. The lower end of the range is Tülu 3's 5×10^{-6} for its 8B model. Overshooting is easy to detect: the model's scores on knowledge benchmarks (MMLU) fall while it gets better at the SFT data's style.

LoRA (Hu et al. 2021) freezes the pretrained weights and trains a low-rank update $\Delta W = BA$ per matrix, with A and B of rank r (16–64 in practice). The trainable parameters are a few percent of the total and the optimizer state is correspondingly small; QLoRA (Dettmers et al. 2023) further stores the frozen weights in 4-bit, so that a 65B model fits on one 48 GB GPU. The cost is quality on large SFT sets, where full fine-tuning is better; on small sets the difference is small.

What SFT cannot do is the point of the rest of the deck. Its loss is the same for a wrong demonstration as for a right one; it maximizes likelihood of whatever it is given. Its data are written by someone else, so the model learns their style and their mistakes. And at every training position the context is the demonstration's own prefix, never the model's: at inference the model conditions on its own earlier tokens, which may be off the demonstrated path, and nothing in SFT taught it to recover. This last effect is called exposure bias, and rejection sampling (slide 6) and RL (Section 3) are the ways around it.

Terms

chat template The function that renders a structured conversation (roles and messages) into the token sequence the model is trained and served on, inserting special tokens for roles, turn ends and tool schemas. Stored with the model's tokenizer.

special token A vocabulary entry that does not correspond to text, such as `<|user|>` or `<|end|>`, used to mark structure. Added to the vocabulary before SFT; its embedding is learned.

loss mask A per-token flag that excludes a position from the loss. SFT masks system, user and tool tokens and trains on assistant tokens only.

full fine-tuning Updating every parameter of the model during SFT. Requires memory for the weights, gradients and optimizer state of the full model.

LoRA Low-Rank Adaptation (Hu et al. 2021): freeze the pretrained weights and train a low-rank additive update BA for each weight matrix. Few trainable parameters, small optimizer state, no change to inference cost once merged.

QLoRA LoRA with the frozen base weights stored in 4-bit (Dettmers et al. 2023), so that a large model can be fine-tuned on one GPU.

packing Concatenating several training examples into one sequence of the context length, separated by end tokens, so that no compute is spent on padding. Attention is masked so examples do not see each other.

epoch One pass over the training set. SFT uses one to three; more causes forgetting.

exposure bias The mismatch between training, where the model conditions on a demonstration's prefix, and inference, where it conditions on its own. Errors at inference lead to contexts never seen in training.

catastrophic forgetting Loss of previously learned ability when a model is trained on new data. In SFT it shows as falling knowledge-benchmark scores as epochs increase.

Intuition

SFT is pretraining with the answer key taped over the questions. The model reads the whole page but is only graded on the part under the tape, so that is the only part it learns to write. Everything that makes SFT easy follows from that (same code, same loss, small data) and so does everything that limits it: it cannot tell a good answer key from a bad one.

2.2 Slide 5: SFT II: where the demonstrations come from

Five years of instruction data: task templates, human demos, synthetic sets, a curated thousand, a million-prompt mix, RL traces.

What the slide shows Six bars on a logarithmic axis of example counts, in date order: FLAN and T0 (2021, task templates, around 10^6 templated examples); InstructGPT (2022, 13k human demonstrations); Self-Instruct and Alpaca (2023, 52k generated by GPT-3); LIMA (2023, 1,000 curated); the Tülu 3 SFT mix (2024, 939k prompts); DeepSeek-R1's stage-3 SFT (2025, 800k traces). Each bar is coloured by the kind of source: academic tasks, human-written, synthetic from a stronger model, curated, mixture, rejection-sampled from RL. A panel gives three lessons: quality, coverage, distillation.

Details The six bars are six answers to the question "who writes the demonstrations?". FLAN (Wei et

al. 2021) and T0 (Sanh et al. 2021) answered: nobody; take existing NLP datasets (62 of them in FLAN, the P3 prompt collection in T0), write a few natural-language templates per task, and render each example as an instruction and an answer (FLAN capped each dataset at 30k examples). This produced instruction following across many tasks at no annotation cost, but the instructions were academic and the answers short. InstructGPT (Ouyang et al. 2022) answered: hired labelers, writing ideal responses to real prompts submitted to the API, 13k of them; expensive, and the standard against which everything after was measured. Self-Instruct (Wang et al. 2022) and Alpaca (Taori et al. 2023) answered: a stronger model; start from 175 seed tasks, ask GPT-3 to generate new instructions and responses, filter for diversity, and get 52k examples for a few hundred dollars. LIMA (Zhou et al. 2023) answered: a thousand examples chosen for quality and variety, enough to turn a 65B base into a competitive chat model, since the base already knows the content and only needs the format. Tülu 3 (Lambert et al. 2024) answered: a mixture, 939k prompts drawn from public sets and new synthetic ones, with sources chosen per skill (math, code, safety, instruction following) and ablated one at a time. DeepSeek-R1 (2025) answered: the model itself, after RL; 800k samples, of which 600k are reasoning traces that passed a correctness check and 200k are general writing and QA.

The three lessons in the panel are the three dimensions of SFT data. Quality: a small number of careful examples is enough for format, and errors in demonstrations are learned as faithfully as correct answers. Coverage: format is not all that SFT teaches; a model that saw no code in SFT will be worse at code than one that did, and a broad mix with per-skill sources outperforms any single set. Distillation: the economics have settled on generating demonstrations with a stronger model and filtering them, because a strong model writes faster and more consistently than annotators and its outputs can be checked by execution or by a judge; most open SFT sets since 2023 are of this kind. The term "distillation" here means SFT on a teacher's outputs, which is the off-policy case of the distillation grid on slide 28.

Terms

instruction tuning SFT on data in the form of an instruction and a response. FLAN introduced the term; the goal was generalization to unseen instructions.

task template A natural-language pattern that turns an example from an existing dataset into an instruction and an answer, e.g. "Translate to French: …". FLAN and T0 used several per task.

Self-Instruct A procedure (Wang et al. 2022) that bootstraps an instruction dataset from a model: generate instructions from seeds, generate responses, filter near-duplicates, repeat.

Alpaca A 52k-example instruction set generated from GPT-3 by Self-Instruct (Taori et al. 2023), and the Llama model fine-tuned on it. The first widely used synthetic SFT set.

LIMA "Less Is More for Alignment" (Zhou et al. 2023): a 65B Llama fine-tuned on 1,000 curated examples competed with models trained on far more, the evidence that SFT teaches mostly format.

Tülu 3 AI2's fully open post-training recipe (Lambert et al. 2024): SFT on a 939k-prompt mix, length-normalized DPO, then RL with verifiable rewards, with all data, code and evaluations released.

curation Selecting training examples by quality, diversity and correctness rather than taking all available data. Done by people for LIMA, by models and filters for most synthetic sets.

distillation (SFT sense) Training a student model on responses generated by a stronger teacher. Also called knowledge distillation from outputs, or sequence-level distillation.

Intuition

The axis is logarithmic, and the bars do not rise with time: LIMA's thousand sits below InstructGPT's thirteen thousand, which sits below Alpaca's fifty-two thousand. The number of examples is not the quantity that improved. What improved is who writes them (a stronger model), how they are chosen (filtered and checked) and how many skills they cover. The recent large sets are large because they cover many skills, not because any one skill needs a million examples.

2.3 Slide 6: SFT III: rejection sampling, the bridge to RL

Sample many answers from the model itself, keep the ones a reward model or verifier likes, fine-tune on those, repeat.

What the slide shows A loop. Prompts go into the policy, which produces K samples per prompt (five are drawn, with scores 0.91, 0.42, 0.77, 0.15, 0.63). A scorer, which may be a reward model, a verifier or a judge, scores them; the best are kept; the model is fine-tuned on the kept samples; the new policy goes back to the start for another round. Below: why it works, the names the same loop has been given, and where it sits in a 2026 recipe.

Details Rejection sampling changes one thing about SFT: the demonstrations are the model's own outputs, selected by a scorer, rather than someone else's writing. That one change removes two of SFT's three limitations. The kept samples are on the model's own distribution, so there is no exposure bias and no style to drift toward: the model is learning to do more often what it already does sometimes. And the scorer supplies the judgment that SFT lacks: wrong samples are not kept. What remains is that the model can only learn from behaviors it can already produce with non-negligible probability, which is also the limitation of RL (slide 18).

The loop is a reinforcement-learning algorithm with a particularly simple update. Sampling K responses and keeping the top-scoring ones is a crude estimate of "responses with above-average reward"; fine-tuning on them with the SFT loss raises their probability. A policy-gradient method would weight each sample by its advantage and update after every batch; rejection sampling binarizes the weight to keep-or-discard and updates after a whole round. That is less sample-efficient but needs no RL infrastructure: generation and training are separate jobs, and any SFT codebase can run it.

The names in the slide are the same loop found in different places. Expert iteration (Anthony et al. 2017, for games) alternates a search that improves on the policy with imitation of the search. STaR (Zelikman et al. 2022) samples rationales for reasoning problems, keeps those that reach the correct answer, and fine-tunes on them, adding a "rationalization" step in which the model is given the answer and asked to produce a rationale for problems it failed. Llama 2 and Llama 3 run rejection sampling with their reward model as the scorer, and Llama 3 made it the main engine of its post-training: six rounds of rejection sampling, SFT and DPO, with no online RL at all. DeepSeek-R1 uses the loop in its third stage, sampling 800k responses from the RL-trained reasoning model, filtering for correctness and readability, and fine-tuning the base model on them; this turns a specialist's behavior into SFT data that can be combined with general data, and it is also how the distillation traces for the small R1 models were produced (slide 17).

Terms

rejection sampling Sampling several responses per prompt from the current model, keeping those a scorer rates highest, and fine-tuning on the kept set. Repeated for several rounds.

on-distribution Describes training data drawn from the model's own output distribution. On-distribution data avoid exposure bias and style drift.

scorer Whatever assigns the score used for selection: a reward model, a verifier, or a judge model. The scorer's errors become the model's errors.

expert iteration The general scheme of alternating a step that improves on the policy (search, sampling plus selection) with a step that imitates the improvement. Rejection-sampling SFT is an instance.

STaR Self-Taught Reasoner (Zelikman et al. 2022): rejection sampling on reasoning problems, keeping rationales that reach the right answer, with a rationalization step for failed problems.

round One pass of the loop: sample, score, select, fine-tune. Llama 3 ran six.

judge A language model prompted to score or compare responses, used as a scorer when no verifier exists and no reward model has been trained.

Intuition

A student who writes five drafts, has a teacher mark the best, and studies that one is learning from their own writing and the teacher's judgment at once. They will not pick up anyone else's style and will not learn from a draft that was wrong. What they cannot do is learn something none of the five drafts contained; for that, more drafts, or a stronger teacher's draft to copy.

3 Preference optimization (slides 7–11)

3.1 Slide 7: RLHF: the original three steps

InstructGPT (2022): demonstrations, then comparisons, then optimization against the learned preference.

What the slide shows Three numbered cards. (1) SFT: labelers write the ideal response to prompts collected from the API; the base model is fine-tuned on them; 13k prompts. (2) Reward model: labelers rank several model outputs per prompt; a copy of the model with a scalar head learns to predict the ranking; 33k prompts. (3) PPO against the RM: the policy samples responses, the reward model scores them, PPO raises the probability of high scorers with a KL penalty to the SFT model; 31k prompts. Below: the result (labelers preferred the 1.3B InstructGPT to the 175B GPT-3), why comparisons, and the lineage from Christiano et al. 2017.

Details InstructGPT (Ouyang et al. 2022) is the paper that fixed the shape of post-training for the following three years, and the three-step diagram is its Figure 2. The prompt counts (13k, 33k, 31k) are from its Table 6; the 33k reward-model prompts yielded many more comparisons, since labelers ranked $K = 4$ to 9 outputs per prompt and every pair of the ranking is a training example. The reward model was a 6B GPT-3 with its unembedding replaced by a scalar head, trained on all the pairs from a prompt in one batch to avoid overfitting to any one.

The result in the slide is the paper's headline: outputs of the 1.3B InstructGPT were preferred by labelers to outputs of the 175B GPT-3, on held-out prompts from the API. A hundredfold difference in size was outweighed by a few tens of thousands of human judgments. The reason is slide 2's: the 175B base model knew more but did not do what was asked; the 1.3B model did. The paper also found that SFT alone closed most of the gap and that RLHF added a further, consistent preference margin, and it introduced the "alignment tax": RLHF models regressed on some public NLP benchmarks, which mixing pretraining gradients into the PPO updates (PPO-ptx) repaired.

Why comparisons rather than more demonstrations? Two reasons the slide gives in one sentence. Judging is easier than writing: a labeler can rank four answers to a question they could not have answered well themselves, and two labelers agree on rankings more often than they would write similar answers. And the reward model turns the judgments into a dense signal: once trained, it scores any response, including ones no labeler saw, so the policy can be optimized on far more responses than were labelled. That second property is also the danger (slide 8): the reward model is being queried off its training data.

The lineage runs from Christiano et al. (2017), who trained Atari and MuJoCo agents from human comparisons of short clips, through Stiennon et al. (2020), who applied the recipe to summarization and found that a 1.3B RLHF model's summaries were preferred to those of a supervised model ten times its size, to InstructGPT, and to Bai et al. (2022), Anthropic's helpful-and-harmless assistant trained with the same loop and the HH-RLHF comparison dataset, which was released and is still used. The algorithm did not change; what grew was the scale of the policy and the breadth of the prompts.

Terms

RLHF Reinforcement learning from human feedback: fit a reward model to human comparisons, then optimize the policy against it with RL, usually with a KL penalty to the starting model.

InstructGPT Ouyang et al. 2022, the OpenAI paper that applied SFT, reward modelling and PPO to GPT-3 on real user prompts. The source of the three-step diagram and of the 1.3B-vs-175B result.

labeler A contracted annotator who writes demonstrations or ranks outputs. InstructGPT used about 40, screened for agreement with the researchers.

scalar head A linear layer that maps the final hidden state (usually at the last token) to one number.

Replacing the language-model head with a scalar head turns the model into a reward model.

ranking An ordering of several responses to a prompt by quality. One ranking of K responses yields $\binom{K}{2}$ pairwise comparisons.

alignment tax The loss of performance on tasks outside the post-training data that a stage can cause. InstructGPT measured it on public benchmarks and reduced it by mixing in pretraining loss.

PPO Proximal Policy Optimization (Schulman et al. 2017), the policy-gradient algorithm used by InstructGPT and most RLHF until 2024. Slide 9.

HH-RLHF Anthropic's helpful-and-harmless comparison dataset (Bai et al. 2022), about 160k human comparisons, released publicly.

Intuition

Writing the best possible answer is hard and slow; saying which of two answers is better is quick and usually reliable. RLHF is a machine for converting the second, cheap kind of judgment into the first, expensive kind of behavior. The reward model is the converter, and like any converter it is accurate only within its operating range.

3.2 Slide 8: Reward models, and why you cannot optimize them forever

A Bradley–Terry model of preferences gives a scalar reward. Push the policy too far against it and true quality falls.

What the slide shows Left: a chosen response ("Rayleigh scattering of blue light") and a rejected one ("The ocean reflects onto the sky") pass through the reward model, a transformer with a scalar head, and receive $r_w = 2.1$ and $r_l = -0.4$. Below, the Bradley–Terry probability and the training loss. Right: a plot after Gao, Schulman and Hilton (2022), reward against $d = \sqrt{\mathrm{KL}(\pi \,\|\, \pi_{\text{init}})}$, the distance the policy has moved. The proxy reward (what is optimized) keeps rising; the gold reward (what was wanted) rises, peaks, and falls. A dashed line marks "stop here" near the peak. The slide labels the curves illustrative.

Details The reward model is trained as a classifier of pairs:

$$P(y_w \succ y_l \mid x) = \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big), \qquad \mathcal{L}_{\text{RM}} = -\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$$

with σ the logistic function. This is the Bradley–Terry model (1952): each item has a latent score, and the probability that one beats another depends only on the difference of scores. Two consequences follow. Rewards are defined only up to an additive constant per prompt, since the constant cancels in the difference; so the reward model's absolute outputs mean nothing, and comparing rewards across prompts is not meaningful without care. And the loss is a logistic regression on the difference of two forward passes, so the reward model trains like a classifier: quickly, from pairs, with accuracy on held-out pairs as its metric (typically 65–75% on human data, since the humans themselves disagree that often).

Data come from people, as in InstructGPT and HH-RLHF, or from a stronger model asked to compare (RLAIF, Lee et al. 2023, who found model labels about as useful as human ones for summarization and dialogue). Sizes run from tens of thousands of pairs to millions. The variants in the slide differ in what the scorer reads and writes: a pairwise classifier takes both responses in one context and outputs a preference directly; a generative reward model writes out a verdict with reasons before giving a score; a rubric judge scores against a checklist written for the prompt. The last two are language models used as scorers, and are the form that extends reward to tasks without a verifier (slide 13).

The right half is the central caution of the whole preference section. Gao, Schulman and Hilton (2022) built a synthetic setup in which a large "gold" reward model stands in for the true human preference, and smaller proxy reward models are trained on labels from the gold one. They then optimized policies against the proxies and measured both. Their fit for the RL case is

$$R_{\mathrm{RL}}(d) = d\,(\alpha - \beta \log d), \qquad d = \sqrt{\mathrm{KL}(\pi \,\|\, \pi_{\mathrm{init}})},$$

where α and β depend on the proxy's size and data. The proxy score rises monotonically with d; the gold score follows the form above, rising, peaking, and declining. The policy has found responses the proxy likes that the gold model does not: it has learned the proxy's errors. More proxy parameters and more comparison data move the peak outward and upward, and the paper gives scaling fits for both. The dashed "stop here" line is what the KL penalty of PPO (slide 9) and the β of DPO (slide 10) implement: a limit on d, chosen so that the policy stays where the proxy is still accurate. The slide calls this "Goodhart's law, measured": a proxy that is optimized stops being a good proxy.

Terms

chosen / rejected The preferred (y_w, "winner") and dispreferred (y_l, "loser") responses in a comparison pair.

logistic function $\sigma(z) = 1/(1+e^{-z})$. Maps a real score difference to a probability.

proxy reward The learned reward model's score, which stands in for the true objective. It is accurate near the data it was trained on and wrong far from it.

gold reward The true objective, or a large reward model standing in for it in an experiment. Observable only in synthetic setups such as Gao et al.'s.

overoptimization The decline of true quality as a policy is optimized further against a proxy reward. Also called reward hacking or Goodharting.

Goodhart's law "When a measure becomes a target, it ceases to be a good measure." The reward model is the measure; RL makes it the target.

KL divergence $\mathrm{KL}(\pi \,\|\, \pi_{\mathrm{ref}}) = \mathbb{E}_{y\sim\pi}[\log \pi(y) - \log \pi_{\mathrm{ref}}(y)]$, a measure of how far one distribution has moved from another. Used throughout post-training as the measure of how far the policy has moved from its starting point.

RLAIF RL from AI feedback (Lee et al. 2023; also Bai et al. 2022): comparisons labelled by a model rather than by people, used to train the reward model.

generative reward model A language model that produces a written judgment and a score, rather than a scalar from a head. Can explain its score and be prompted with a rubric.

rubric A written checklist of properties a good response should have for a given prompt, used by a judge to score. Rubric rewards extend RL to tasks with no verifier.

Intuition

A student who studies only the past exam papers will do better on the next exam, up to a point, and then start doing worse, because they have begun to learn the examiner's habits rather than the subject. The reward model is the past papers. The KL penalty is a rule that says: study, but do not become unrecognizable to the teacher who wrote the papers.

3.3 Slide 9: PPO for language models

Policy gradient with a clipped ratio, a KL leash to the reference model, and a value model: four networks in memory.

What the slide shows A dataflow with four networks. The prompt enters the policy π_θ, which produces a response y; the reward model r_ϕ scores it; the reference model π_{ref} gives the KL term; the value model V_ψ gives the advantage; the PPO update with a clipped ratio moves the policy's parameters, and the loop repeats. Below: the clipped objective, the ratio ρ_t, and the per-token reward. A panel lists the knobs that matter and the failure modes.

Details The objective is

$$\mathscr{L}_{\text{PPO}} = \mathbb{E}_t\Big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big], \qquad \rho_t = \frac{\pi_\theta(y_t \mid x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid x, y_{<t})},$$
$$r_t = r_\phi(x,y)\mathbb{1}[t=T] - \beta\,\text{KL}_t, \qquad \text{KL}_t = \log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t}).$$

Read it in three parts. The ratio ρ_t compares the current policy's probability of the sampled token with the probability under the policy that generated the sample; at the first optimizer step after sampling it is 1, and it drifts as the parameters move. Multiplying ρ_t by the advantage $\hat{A}_t$ is the importance-weighted policy gradient: raise the probability of tokens with positive advantage, lower it for negative. The clip and the min are PPO's contribution (Schulman et al. 2017): if the ratio has moved outside $[1-\varepsilon, 1+\varepsilon]$ in the direction the advantage favours, the gradient is cut off, so that a few optimizer steps per batch of samples cannot move the policy far. This makes it safe to reuse each batch for several updates, which is what makes PPO sample-efficient enough for language models, where each sample costs a full generation.

The reward is assembled per token. The reward model gives one scalar for the whole response, placed at the last token. The KL term is subtracted at every token, which is how the leash is applied: tokens on which the policy has drifted far from the reference are penalized wherever they occur. The coefficient β sets the leash length and is often adapted during training to hold a target KL. The value model V_ψ predicts the expected return from each position and is trained alongside the policy; the advantage $\hat{A}_t$ is the return minus that prediction, computed with generalized advantage estimation over the token sequence. Its purpose is variance reduction: the gradient is weighted by how much better the outcome was than expected, not by the raw reward.

Four networks is the practical burden. The policy and the value model train, so each needs weights, gradients and optimizer state; the reference and reward models are frozen but must run a forward pass on every sample. At 70B parameters this is four 70B networks plus the optimizer state of two, and a generation engine besides. DPO (slide 10) removes the reward and value models and the sampling; GRPO (slide 12) removes the value model and keeps the sampling. Both exist because of this slide's memory bill.

The panel's knobs are InstructGPT's and the open re-implementations' defaults: $\varepsilon = 0.2$; β from 0.01 to 0.1; a few optimizer epochs per batch of rollouts. The failure modes are the ones the next slides return to: reward hacking (slide 8), answer length growing because the reward model prefers longer answers, and the value model lagging behind a policy that moves quickly, which makes the advantages noisy.

Terms

importance ratio ρ_t, the ratio of the current policy's probability of a sampled token to the sampling policy's. It corrects for the samples having been drawn from a slightly older policy.

clipping Limiting the importance ratio to $[1-\varepsilon, 1+\varepsilon]$ in the objective, so that one batch of samples cannot move the policy far. $\varepsilon = 0.2$ is standard.

advantage $\hat{A}_t$, how much better the outcome was than expected at position t: the return minus a baseline. The policy gradient is weighted by it.

value model A network V_ψ trained to predict the expected reward from a position in the response; its prediction is the baseline for the advantage. Also called the critic.

reference model π_{ref}, the frozen starting policy (usually the SFT model) that the KL penalty measures distance from.

generalized advantage estimation (GAE) A way of computing advantages from per-token rewards and value predictions that trades bias for variance with a parameter λ. Standard in PPO.

rollout A response sampled from the current policy during RL, together with its rewards and log-probabilities. One batch of RL consists of many rollouts.

on-policy Training on samples drawn from the current policy. PPO is approximately on-policy: samples are reused for a few steps and the clip keeps the policy close.

KL coefficient β, the weight of the KL penalty. Larger β keeps the policy nearer the reference.

Intuition

The policy is a student who tries answers and is scored; the value model is their own guess at how well they are doing, so that only surprises move them; the clip stops them from changing their mind too far on one batch of feedback; the reference model is the person they were before the course, whom they are not allowed to stop resembling. Four people in the room for one student is the cost, and the reason the next two slides exist.

3.4 Slide 10: DPO: skip the reward model

The KL-regularized objective has a closed-form optimum; put into Bradley–Terry, it gives a loss on the policy alone.

What the slide shows A four-step derivation chain. (1) The objective: maximize expected reward minus β times the KL to the reference. (2) Its optimum: $\pi^*(y \mid x) = \pi_{\text{ref}}(y \mid x) e^{r(x,y)/\beta} / Z(x)$. (3) Invert: $r(x,y) = \beta \log(\pi^*/\pi_{\text{ref}}) + \beta \log Z(x)$. (4) In Bradley–Terry, $Z(x)$ cancels in the difference $r(x,y_w) - r(x,y_l)$. Then the DPO loss. On the right, the two pipelines compared (RLHF: SFT, RM, PPO, four models and samples; DPO: SFT, DPO, two models, no samples, off-policy) and a panel on variants and the catch.

Details Direct Preference Optimization (Rafailov et al. 2023) starts from the same objective PPO optimizes, written for whole responses:

$$\max_\pi \; \mathbb{E}_x \mathbb{E}_{y\sim\pi(\cdot|x)} \big[r(x,y) \big] - \beta \, \mathrm{KL}\big(\pi(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x)\big).$$

For a fixed prompt this is a maximization over a distribution, and it has an exact solution: $\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \exp(r(x,y)/\beta)$, the reference reweighted by the exponentiated reward, with $Z(x) = \sum_y \pi_{\text{ref}}(y \mid x) e^{r(x,y)/\beta}$ the normalizer. The derivation is a few lines of calculus of variations (Exercise 5 in Appendix C). Step 3 inverts the relationship: any reward can be written as β times the log-ratio of its optimal policy to the reference, plus a term that depends only on the prompt. Step 4 is the observation that makes it useful: the Bradley–Terry probability depends only on the reward difference between two responses to the same prompt, and the $\beta \log Z(x)$ term is the same for both, so it cancels. Substituting into the reward-model loss of slide 8 gives

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$

This is the reward-model loss with the reward model replaced by the policy's own log-ratios. Training it fits a reward model implicitly, $\hat{r}(x,y) = \beta \log(\pi_\theta / \pi_{\text{ref}})$, and the policy that fits it is, by construction, the optimal policy for that reward under the KL constraint. The same comparison data, two forward passes per response (policy and reference), a classification loss, and no sampling.

The right side is the engineering consequence. RLHF needs four networks and a generation loop; DPO needs two, and the reference's log-probabilities can be precomputed once, leaving one. Any SFT codebase can run it. β plays the same role as PPO's KL coefficient, with 0.1 a common default; the Tülu 3 value of 5 is for the length-normalized variant, where the log-ratios are divided by the response length and the scale changes.

The variants each remove or change one piece. IPO (Azar et al. 2023) replaces the logistic loss by a bounded squared one, so that the policy cannot drive the margin to infinity on pairs it is sure of. KTO (Ethayarajh et al. 2024) works from unpaired labels (this response was good, that one bad), which are easier to collect. SimPO (Meng et al. 2024) drops the reference model and normalizes by length, using the average log-probability as the implicit reward. ORPO (Hong et al. 2024) adds an odds-ratio preference term to the SFT loss, so that SFT and preference are one stage.

The catch is in the word off-policy. The pairs were sampled by some model, often not the one being trained, and once the policy moves they are not samples from it. The implicit reward is then being trusted on responses the policy would never produce, and nothing checks it there: the overoptimization of slide 8 applies, without even a proxy model to inspect. Iterative or online DPO resamples pairs from the current policy each round, scored by a reward model or judge, which restores the on-policy property at the cost of the sampling DPO was meant to avoid. By 2026 most frontier open recipes have moved to RL for the same reason (slide 11).

Terms

DPO Direct Preference Optimization (Rafailov et al. 2023): train the policy on comparison pairs with a classification loss on its log-ratios to the reference, with no reward model and no sampling.

closed-form optimum The exact maximizer of the KL-regularized reward objective, $\pi^* \propto \pi_{\text{ref}} e^{r/\beta}$. It exists for any reward but cannot be sampled from directly because of the normalizer.

partition function $Z(x)$, the normalizer of the optimal policy, a sum over all responses. Intractable, and the reason the inversion in step 3 would be useless without the cancellation in step 4.

implicit reward $\beta \log(\pi_\theta(y \mid x)/\pi_{\text{ref}}(y \mid x))$, the reward that DPO's trained policy is optimal for. Can be read off any pair of policy and reference.

off-policy Training on data not sampled from the current policy. DPO is off-policy: the pairs come from whatever model produced them.

IPO Identity Preference Optimization (Azar et al. 2023): a bounded, squared-error version of the DPO objective that does not overfit to deterministic preferences.

KTO Kahneman–Tversky Optimization (Ethayarajh et al. 2024): preference training from unpaired good/bad labels, with a loss modelled on prospect theory.

SimPO Simple Preference Optimization (Meng et al. 2024): DPO without a reference model, using the length-normalized log-probability as the reward and a target margin.

ORPO Odds Ratio Preference Optimization (Hong et al. 2024): SFT loss plus an odds-ratio penalty on the rejected response, combining SFT and preference in one stage without a reference.

iterative DPO DPO repeated over rounds, with new pairs sampled from the current policy and labelled by a reward model or judge each round, so the data stay near on-policy.

Intuition

RLHF fits a judge and then trains the student to please the judge. DPO notices that the best student for any judge has a particular form, and fits that form to the comparisons directly, so the judge never has to be built. The trick is exact; the cost is that the student is only ever examined on the comparisons it was given, not on what it would say itself.

3.5 Slide 11: Choosing a preference method

What each needs, what it costs, and who used what. The trend from 2024 to 2026 is toward on-policy methods.

What the slide shows A table of five methods against six properties: whether a reward model and a value model are needed, whether the method is on-policy, how many networks are in memory, what the signal is, and typical use. DPO: no, no, no, 2, pairs; chat preferences in Llama 3, Tülu 3, OLMo 3. SimPO and ORPO: no, no, no, 1, pairs; a cheap preference stage. PPO: yes, yes, yes, 4, scalar per response; InstructGPT and Llama 2, now rare in open recipes. RLOO and REINFORCE: RM or verifier, no, yes, 2–3, scalar with a leave-one-out baseline. GRPO: RM or verifier, no, yes, 2–3, scalar group-normalized; reasoning RL in R1, Qwen3 and most RLVR since. Below: a rule of thumb, what the open recipes did, and a panel on DPO receding in 2026.

Details The table's columns are the costs and the capabilities. A reward model costs a training run and comparison data, and introduces the proxy of slide 8; a verifier costs nothing but exists only for checkable tasks. A value model costs a second trained network and the lag problem of slide 9. On-policy methods need a generation engine in the training loop and spend most of their compute on sampling, but learn from the policy's own behavior; off-policy methods learn from a fixed dataset, cheaply, and drift off the data as they train. The number of networks is the memory bill.

RLOO (Ahmadian et al. 2024) deserves a line because it shows how little of PPO is needed for language models. It is REINFORCE, the plain policy gradient $\nabla \log \pi(y \mid x)\,(r - b)$, with the baseline b for each sample computed as the mean reward of the other $K - 1$ samples for the same prompt. No value model, no per-token credit assignment, one reward per response; the authors found it competitive with PPO on RLHF benchmarks. GRPO (slide 12) is the same idea with the baseline and a standard deviation taken over the group, plus PPO's clipping.

The rule of thumb is the practical content. Use DPO when the data are pairs and there is no verifier: style, helpfulness, safety preferences, where the signal is a human or model judgment and the behavior is near the SFT model already. Use GRPO or RLOO when a response can be scored by a program (math answers, unit tests, instruction constraints), or when a strong reward model exists and the budget allows sampling, because then the search for better responses pays. The recipes in the slide follow it: Llama 3 (2024) used only rejection sampling, SFT and DPO, repeated six rounds, and reported choosing DPO over PPO because it needed less compute at that scale and performed better, especially on instruction following (IFEval); Tülu 3 ran SFT, DPO, then RLVR; OLMo 3 (2025) ran the same three stages for each of its variants.

The 2026 panel is flagged as coming from write-ups and commentary rather than ablations, and the flag should be kept. Several 2026 open releases (DeepSeek-V4, GLM-5, Kimi K2.5, Nemotron 3) describe recipes centred on RL with verifiable and rubric rewards, and several report no DPO stage: the preference signal enters as a reward model or judge inside the RL. Whether a separate DPO stage still helps is an empirical

question that only ablations answer, and where one was published (Nemotron 3) it did. The honest summary is that DPO is receding as the main engine and persisting as a cheap stage.

Terms

REINFORCE The basic policy-gradient estimator (Williams 1992): $\nabla \log \pi(y \mid x)\,(r(x,y) - b)$ with a baseline b to reduce variance.

RLOO REINFORCE Leave-One-Out (Ahmadian et al. 2024): sample K responses per prompt and use the mean reward of the other $K-1$ as each one's baseline. No value model.

GRPO Group Relative Policy Optimization (Shao et al. 2024): PPO's clipped objective with advantages computed by normalizing each response's reward within its group of samples for the same prompt. Slide 12.

baseline A quantity subtracted from the reward before weighting the policy gradient, to reduce variance without changing the expected gradient. A value model, a group mean, or a leave-one-out mean.

Llama 3 Meta's 2024 open-weight family (Grattafiori et al. 2024); its post-training ran six rounds of rejection sampling, SFT and DPO with no online RL.

OLMo 3 AI2's fully open model family (November 2025), with Think, Instruct and RL-Zero variants, each post-trained by SFT, DPO and RLVR.

ablation An experiment that removes one component of a recipe and measures the change, the only way to know whether a stage helps. Release notes rarely include them.

Intuition

The question to ask of any preference method is: who produced the responses being learned from, and who scored them? If someone else produced them (DPO), training is cheap and the method cannot wander far from that data, for better and worse. If the model produced them (RL), the method can find new behavior, and the scorer had better be right.

4 Reinforcement learning (slides 12–14)

4.1 Slide 12: GRPO: group-relative advantages, no value model

Sample a group of answers per prompt, score them, and use each answer's standing in its group as the advantage.

What the slide shows A prompt enters the policy, which produces five samples $y_1 \dots y_5$. Each is scored once, as a whole, by a verifier or reward model: rewards 1, 0, 1, 0, 0. The advantages are computed from the group: mean 0.4, standard deviation 0.49, so the two correct answers get $\hat{A} = +1.22$ and the three wrong ones -0.82. Below, the GRPO objective. Two notes say what is gone (the value model) and what is kept (PPO's clipped ratio and a KL term). A panel gives the origin and fit.

Details The advantage is

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_G)}{\text{std}(r_1, \dots, r_G)},$$

the same number for every token of response i. With rewards $(1,0,1,0,0)$ the mean is 0.4 and the population standard deviation is $\sqrt{0.4 \cdot 0.6} = 0.49$, giving $(1-0.4)/0.49 = 1.22$ and $(0-0.4)/0.49 = -0.82$ as on the slide. The group plays the role of the value model: the expected reward for this prompt is estimated by the mean of the siblings, and "better than expected" is measured against them. The normalization by the standard deviation puts prompts of different difficulty on the same scale, which is convenient and, as slide 14 notes, not unbiased.

The objective is

$$\mathcal{J}_{\text{GRPO}} = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|y_i|}\sum_{t=1}^{|y_i|}\min\big(\rho_{i,t}\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\big) - \beta\,\mathbb{D}_{\text{KL}}\big[\pi_\theta \,\|\, \pi_{\text{ref}}\big]\Big]$$

with $\rho_{i,t}$ the importance ratio of slide 9. Three things differ from PPO. The advantage is per response, not per token, so every token of a correct answer is pushed up equally; there is no credit assignment within a response. The loss is averaged per response over its tokens $(1/|y_i|)$ and then over the group $(1/G)$. And the KL penalty is a separate term in the objective, estimated by the unbiased estimator $\pi_{\text{ref}}/\pi_\theta - \log(\pi_{\text{ref}}/\pi_\theta) - 1$ at each sampled token, rather than subtracted from the reward as in PPO.

What is gone is the value model and with it a trained network, its optimizer state, and the lag problem; the compute saved goes into sampling a larger group. Two or three networks remain: the policy, the reference (if $\beta > 0$), and the reward model (if the reward is not from a verifier). What is kept is the clipped ratio, which allows several optimizer steps per batch of rollouts, and the leash.

GRPO was introduced in DeepSeekMath (Shao et al. 2024) for math reasoning and became the engine of DeepSeek-R1 and of most reasoning RL in 2025–26. It fits a sequence-level reward: when the only signal is "right or wrong" at the end, a per-token value model has little to predict from, and the group estimate is as good. It fits dense per-token rewards poorly, since the single advantage per response throws away their structure. Group sizes are 8 to 64. A prompt on which all G samples get the same reward has zero advantage for every sample and contributes no gradient; such prompts are wasted compute, and recipes filter them (slide 14).

Terms

group The G responses sampled from the policy for one prompt in one RL step. Advantages are computed within the group.

group-relative advantage A response's reward minus the group mean, divided by the group standard deviation. The signal GRPO trains on.

sequence-level reward One scalar for the whole response, assigned after it is complete. Verifiable rewards are sequence-level.

credit assignment Deciding which tokens or steps of a response were responsible for its reward. GRPO does none within a response; every token gets the response's advantage.

KL estimator A sample-based approximation of the KL divergence. GRPO uses $\pi_{\text{ref}}/\pi_\theta - \log(\pi_{\text{ref}}/\pi_\theta) - 1$ per token, which is non-negative and unbiased.

DeepSeekMath Shao et al. 2024, the paper that introduced GRPO while training a 7B math model, and whose Section 5.2.1 is a unified view of SFT, rejection sampling, DPO, PPO and GRPO as one gradient with different weights.

sampling budget The number of responses generated per RL step, G times the number of prompts. In GRPO it is where most of the compute goes.

Intuition

Five students answer the same question; two are right. Instead of a tutor who estimates how hard the question was, each student is graded against the other four: the right ones did better than the group, the wrong ones worse. If all five agree, right or wrong, there is nothing to learn from the comparison. The class is the critic.

4.2 Slide 13: RL with verifiable rewards (RLVR)

When a program can check correctness, the reward model disappears and there is far less to hack.

What the slide shows A scatter of reward sources on two axes: coverage (how many tasks the source can score) against reliability of the signal. In the top-left, verifiable: math answer check, unit tests, format and constraint checks (IFEval). Lower and to the right: rubric judge, reward model, LLM judge on free text. The placement is labelled illustrative. On the right, three verifiable rewards spelled out: math (1 if the normalized boxed answer equals the gold answer, else 0, often plus a format reward), code (run hidden unit tests in a sandbox; reward the fraction passed or 1 only if all pass), instructions (check each constraint by program). Below: the name's origin in Tülu 3, and the ways RLVR is still hackable.

Details A verifiable reward is one computed by a program from the response and a reference, with no learned model in the loop. For a math problem with a known answer, the response is parsed for a final answer (in a `\boxed{}` or between `<answer>` tags), normalized (strip whitespace, canonicalize fractions and units), and compared to the gold answer; the reward is 1 or 0. For a coding task, the response's code is run in a sandbox against hidden tests; the reward is the fraction of tests passed, or 1 only if all pass, the latter being harder to game. For an instruction with checkable constraints, in the style of IFEval (Zhou et al. 2023), each constraint ("exactly three bullet points", "no commas", "end with a question") is checked by a function and the reward is the fraction or the conjunction. A format reward, usually small, is added for producing the expected structure (think tags, answer tags), so that the parser can find the answer.

The name is from Tülu 3 (Lambert et al. 2024), which ran a final RL stage against exactly these three rewards on GSM8K, MATH and IFEval prompts after its DPO stage, and found gains on those three

benchmarks. DeepSeek-R1-Zero (2025) used only an accuracy reward and a format reward, on a base model, and is the demonstration that this signal alone suffices for long reasoning to emerge (slide 16). Verifiable rewards are what moved RL from the last step of the recipe to the centre: they are exact, cost nothing per sample beyond running the checker, and cannot be optimized past their validity in the sense of slide 8, since a correct answer is correct no matter how far the policy has moved. The leash can be loosened (β to zero, as in DAPO) and the run extended for thousands of steps.

The two axes of the plot are the trade-off that the right side of the slide works through. Verifiable rewards are reliable but cover only tasks where a checker exists: math with a single answer, code with tests, constraints that are mechanical. Reward models and judges cover everything and are reliable in proportion to how far the policy has moved from their training data. Rubric rewards sit between: a judge model scores against a written checklist for the prompt (Kimi K2's self-critique rubric is the example on the slide), which is more reliable than free-form judging because the criteria are explicit, and more general than a verifier because the criteria can be written for any task. Generative reward models, which write a verdict before scoring, are the same move. The 2026 recipes (slide 27) use verifiable rewards wherever they exist and rubric rewards for the rest.

"Still hackable at the edges" lists what a program-checked reward does not prevent. A policy with write access to the test file edits the tests. A policy that is unsure of the answer writes several candidates in several formats and lets a lenient parser find the right one. A policy rewarded for constraint satisfaction produces three bullets of nonsense. The mitigations are the ones the slide names: sandboxes that the policy cannot escape, canonical extraction that accepts one answer only, and a judge for the quality a checker cannot see. The reward is only as good as the checker, and the policy will find the checker's gaps.

Terms

RLVR Reinforcement learning with verifiable rewards: RL in which the reward is computed by a program (answer check, tests, constraint check) rather than by a learned model. Named by Tülu 3.

gold answer The reference answer for a problem, against which the response's extracted answer is compared.

answer extraction Parsing the final answer out of a response, from a `boxed` expression or answer tags, and normalizing it before comparison.

format reward A small reward for producing the expected structure (think and answer tags, a boxed answer), so that the extractor can find the answer and the policy learns the protocol.

sandbox An isolated execution environment in which the policy's code is run against tests, with no access to the tests, the network or the host.

IFEval Instruction-Following Evaluation (Zhou et al. 2023): prompts with verifiable constraints (length, format, keywords) checked by code. Used both as a benchmark and as an RLVR reward.

coverage The range of tasks a reward source can score. Verifiers have low coverage and high reliability; judges the reverse.

rubric reward A score from a judge model against a written checklist of criteria for the prompt. Extends RL beyond verifiable tasks with more control than free-form judging.

self-critique A model scoring its own (or a sibling's) outputs against a rubric, used by Kimi K2 as a reward for non-verifiable tasks.

reward hacking Any way a policy raises its reward without doing what the reward was meant to measure: editing tests, gaming the parser, satisfying constraints with nonsense.

Intuition

A reward model is a teacher's opinion; a verifier is the answer key. You can argue a teacher into a better grade, and a model optimized against a teacher learns to argue. You cannot argue with the answer key, so the model has to actually get the answer, and it can be pushed as hard as the budget allows. The limit is that answer keys exist only for some subjects, and that a determined student will still look for ways to copy the key.

4.3 Slide 14: The RL loop at scale, and what breaks

Most of the compute is generation; the trainer and the inference engine run side by side.

What the slide shows Four boxes in a row: the trainer (FSDP or Megatron, holding policy weights and optimizer state), the rollout engine (vLLM or SGLang, sampling G responses per prompt), the verifiers (sandboxes, checkers, judge models), and the environments. Weights flow from trainer to engine and samples back; responses flow to the verifiers and rewards back. A note says the loop is asynchronous: generation runs a step or two ahead of the trainer, and stale samples are reweighted or filtered. Three cards describe what breaks: train/inference mismatch, entropy collapse and length, biased estimators.

Details The architecture follows from the arithmetic. One RL step samples, say, 512 prompts times 16 responses times several thousand tokens: tens of millions of generated tokens, each a forward pass at batch size one over a long and growing context. The gradient step that follows touches the same tokens once. Generation is therefore most of the wall-clock time, and it is a serving problem, not a training problem: it wants the continuous batching, paged KV cache and optimized kernels of an inference engine. So the standard design (HybridFlow/veRL, Sheng et al. 2024; OpenRLHF; slime) runs a training framework and an inference engine side by side, ships the policy weights to the engine after each update, and ships the samples back. Verification runs in separate workers: thousands of sandboxed test runs per step for code, judge-model calls for rubrics.

Synchronous operation, in which the trainer waits for all rollouts and the engine waits for the update, leaves one side idle at all times, and the longest rollout in the batch sets the step time. Asynchronous operation lets the engine keep sampling from a slightly stale policy while the trainer updates; the samples are then one or two policy versions old, which is the off-policy situation the importance ratio of slide 9 corrects for, within limits. Recipes cap the staleness and either reweight stale samples by the ratio or drop those too far off.

The three cards are the three failures that every large RL run of 2025–26 has reported. First, train/inference mismatch: the engine and the trainer compute the same model with different kernels, batch sizes and numerics, and the token probabilities they report differ slightly; the ratio ρ is then computed between two different implementations of the same policy, and the clip is applied to noise. The fixes are to recompute the log-probabilities in the trainer before the update, to apply importance weights between engine and trainer probabilities, or to mask the tokens where they disagree most (IcePop, from the Ring-1T report). Second, entropy collapse: as the policy concentrates on the responses that have been rewarded, its per-token entropy falls, the group of G samples becomes near-identical, and exploration stops; at the same time responses grow longer, because length is rewarded indirectly. DAPO (Yu et al. 2025) addresses both with four changes: clip-higher (a larger upper clip bound, 0.28 against 0.2, so that low-probability tokens can be raised more), dynamic sampling (drop prompts on which all G agree and refill the batch), a token-level loss (sum over all tokens in the batch rather than per-response averaging), and a soft overlong penalty. Third, biased estimators: Dr. GRPO (Liu et al. 2025) shows that the $1/|y_i|$ normalization in GRPO gives shorter responses a larger per-token weight, so that long wrong answers are penalized less per token than short wrong ones and length grows under a negative advantage; and that dividing by the group standard deviation up-weights prompts that are nearly always right or nearly always wrong. Removing both terms gives an unbiased policy gradient and, in their experiments, the same accuracy with shorter outputs.

Terms

trainer The process that holds the policy's parameters and optimizer state and performs the gradient update, sharded across GPUs with FSDP or Megatron.

rollout engine An inference server (vLLM, SGLang) used inside the training loop to sample responses from the current policy at serving speed.

FSDP Fully Sharded Data Parallel: a PyTorch scheme that shards parameters, gradients and optimizer state across GPUs and gathers them layer by layer during compute.

weight sync Copying the updated policy parameters from the trainer to the rollout engine after each step. Its cost is one reason to run several optimizer steps per batch.

asynchronous RL A loop in which generation and training overlap, so that samples are produced by a policy one or more updates old. Faster; requires staleness control.

staleness The number of policy updates between the version that generated a sample and the version being trained on it.

train/inference mismatch The difference between the token probabilities reported by the rollout engine and by the trainer for the same policy, due to different kernels and numerics.

IcePop A fix for the mismatch (Ring-1T report, 2025): mask out, in the loss, the tokens on which the engine's and the trainer's probabilities disagree beyond a threshold.

entropy collapse The fall of the policy's per-token entropy during RL until its samples are near-identical and exploration stops.

DAPO Decoupled Clip and Dynamic Sampling Policy Optimization (Yu et al. 2025): GRPO with clip-higher, dynamic sampling, a token-level loss and a soft length penalty, with $\beta = 0$.

clip-higher Using a larger upper clip bound ($1 + \varepsilon_{\text{high}}$ with $\varepsilon_{\text{high}} = 0.28$) than lower, so that rare tokens can gain probability faster. Counteracts entropy collapse.

dynamic sampling Discarding prompts on which all G samples received the same reward, which carry no gradient, and sampling more prompts to fill the batch.

Dr. GRPO "GRPO Done Right" (Liu et al. 2025): GRPO with the per-response length normalization and the group-standard-deviation normalization removed, giving an unbiased gradient and no length growth.

overlong penalty A reward penalty applied to responses that exceed a length budget, soft (graded) in DAPO, to stop length growth without truncating.

Intuition

An RL run is a factory with a slow assembly line (generation) and a fast inspection station (the gradient step). Everything about the design is about keeping the line moving: run it on serving hardware, let it run ahead, accept slightly old parts. The three failures are the three ways the inspection goes wrong: the line and the station measure parts differently; the line starts making only one kind of part; the station's scale is miscalibrated by part length.

5 Reasoning (slides 15–18)

5.1 Slide 15: Thinking tokens: spend compute at inference

Models that write out their reasoning improve with more thinking tokens; RL taught them to use them.

What the slide shows A plot, labelled illustrative, of accuracy on hard math against thinking tokens per problem on a log axis. A reasoning model's curve rises from about 15% at a hundred tokens to near its ceiling above ten thousand; the same base model answering directly is a flat dashed line near 18%. A panel traces how we got here: chain-of-thought prompting (2022), bootstrapping with STaR (2022), process supervision (2023), test-time compute (2024), o1 and DeepSeek-R1 (2024–25).

Details The plot is the test-time scaling curve: for a model trained to reason in writing, accuracy on hard problems rises roughly linearly in the logarithm of the tokens it is allowed to spend, up to the problem's ceiling, and each model has its own curve. The curve is the reason reasoning models exist. Before 2024, the way to make a model better at a hard problem was to make the model bigger; the curve says there is a second axis, inference compute, along which the same model gets better, and that the exchange rate can be favourable.

The panel's history is five steps. Chain-of-thought prompting (Wei et al. 2022) showed that asking a large model to "think step by step", or giving it a few worked examples with reasoning, raised accuracy on arithmetic and reasoning benchmarks, and that the effect appeared only above a certain model size: the ability was latent in the base model and only needed to be elicited. STaR (Zelikman et al. 2022) turned elicited reasoning into training data by rejection sampling on correct answers (slide 6). Process supervision (Lightman et al. 2023) trained a reward model, on the PRM800K dataset of 800k step-level human labels, to score each step of a solution rather than only the final answer, and found it a better selector than outcome supervision on MATH; it also raised the question, still open, of whether step-level rewards are worth their labelling cost when outcome rewards are free. Snell et al. (2024) measured the exchange rate: with inference compute spent well (revising answers, searching over steps with a verifier, allocated per problem by difficulty), a small model could beat one 14 times larger on problems where the small model was not hopeless. Then o1 (OpenAI, September 2024) showed a model that had learned, through RL, to produce long private reasoning before answering, with accuracy that rose with both training compute and thinking length; the methods were not published. DeepSeek-R1 (January 2025) published a recipe that produced the same behavior from an open base model, and the open community reproduced it within weeks.

The word "thinking" is used throughout for the tokens the model produces between the question and its final answer, usually enclosed in tags and often hidden from the user. They are ordinary tokens; what is special is that the model was trained, by RL, to produce them in the quantity and form that raises its accuracy, rather than imitating a demonstrated chain.

Terms

chain of thought (CoT) A sequence of intermediate reasoning steps written out in text before the final answer. Elicited by prompting in 2022; trained by RL since 2024.

thinking tokens The tokens a reasoning model generates before its answer, usually between `<think>` tags and hidden from the user by the interface. Their count is the test-time compute spent.

test-time scaling The rise of accuracy with inference compute (thinking length, number of samples, search) for a fixed model. The second axis, beside model size.

process supervision Training a reward model on labels for each step of a solution (a process reward model, PRM), rather than for the final answer only.

PRM800K Lightman et al.'s 2023 dataset of about 800k step-level correctness labels on MATH solutions, used to train process reward models.

outcome supervision Reward for the final answer only. Verifiable rewards are outcome supervision.

o1 OpenAI's September 2024 model, the first deployed reasoning model trained by RL to think at length before answering. Its methods were described only in a blog post.

DeepSeek-R1 DeepSeek's January 2025 reasoning model and paper, which published a complete recipe (R1-Zero and the four-stage R1) and released the weights.

latent ability A capability present in the base model's distribution but not produced by default, which post-training elicits rather than creates. Chain-of-thought prompting was the first clear evidence.

Intuition

Before 2024, a model's answer was the first thing it thought of. A reasoning model is allowed to think out loud, try something, notice it is wrong, and try again, and the longer it is allowed, the better its final answer, up to a point. What RL added is not the ability to think in words, which prompting had already found, but the habit of doing it well without being asked.

5.2 Slide 16: DeepSeek-R1: RL makes the model think longer

R1-Zero: a base model, GRPO and two rule rewards; the reasoning behaviors appeared on their own.

What the slide shows Left, a plot after R1-Zero's Figure 3 (labelled illustrative): average response length against RL steps, rising from under a thousand tokens to about ten thousand over eight thousand steps, with a note that re-checking, backtracking and "wait" appear along the way. Below it, a description of R1-Zero: DeepSeek-V3-Base, GRPO, reward equal to accuracy plus format; response length and AIME accuracy rose together; self-verification and reflection emerged, the paper's "aha moment"; flaws were language mixing and poor readability. Right, the four stages of R1 proper: cold-start SFT, reasoning RL, rejection-sampling SFT, RL for all scenarios. A note says the shape, two SFT and two RL stages alternating, recurs in Qwen3 and GLM-5.

Details R1-Zero is the experiment that settled a question: can RL with verifiable rewards alone, starting from a base model with no SFT, produce long reasoning? The setup was DeepSeek-V3-Base (671B total, 37B active), GRPO with a KL term, and two rule-based rewards: accuracy (the boxed answer matches, or the code passes tests) and format (the reasoning is inside think tags). No reward model, no demonstrations of reasoning. Over training, the average response length grew steadily, as the plot sketches, because longer responses that re-check and backtrack were more often correct and were therefore reinforced; AIME 2024 pass@1 rose from 15.6% to 71.0% (86.7% with majority voting over 64 samples), matching o1-0912. The paper's "aha moment" is a training sample in which the model stops mid-solution, writes "Wait, wait. Wait. That's an aha moment I can flag here," and re-derives the step: a behavior no one demonstrated, selected because it helped. The flaws were that R1-Zero mixed languages within a response and wrote reasoning no one would want to read, which is what the full R1 recipe fixes.

R1 adds data and two more stages around the same RL. Stage 1, cold-start SFT: a few thousand long, readable chain-of-thought examples, some from R1-Zero's outputs cleaned up, some from few-shot prompting, formatted with a summary after the reasoning; the purpose is to fix the format and readability before RL, so that the RL does not have to discover them. Stage 2, reasoning RL: the same GRPO on math, code and logic, with a language-consistency reward added to stop the mixing (which slightly lowered accuracy and was kept for readability). Stage 3, rejection-sampling SFT: the stage-2 model samples solutions, about

600k correct and readable reasoning traces are kept, 200k general examples (writing, QA, translation, from the V3 pipeline) are added, and the *base* model is fine-tuned on the 800k for two epochs. Stage 4, RL for all scenarios: rule rewards on the reasoning prompts, reward models for helpfulness (judged on the final summary) and harmlessness (judged on the whole response) on the rest.

The shape is the point of the slide's final note. RL discovers a behavior on the tasks where it can be rewarded; SFT on the discovered behavior consolidates it, broadens it to a general data mix, and transfers it to a clean starting point; a second RL polishes across all tasks. Qwen3's four stages (slide 17) and GLM-5's recipe have the same alternation, and the distilled small models (slide 17) are the stage-3 data applied to a different base.

Terms

R1-Zero DeepSeek's RL-only reasoning model: V3-Base trained by GRPO on accuracy and format rewards with no SFT. The evidence that long reasoning emerges from verifiable rewards alone.

DeepSeek-V3-Base The 671B-total, 37B-active mixture-of-experts base model (December 2024) from which R1-Zero and R1 were trained.

accuracy reward A rule-based reward of 1 if the extracted answer matches the gold answer (or the code passes), else 0.

aha moment The paper's name for the appearance, during R1-Zero's training, of self-verification and backtracking in the model's reasoning without any demonstration of them.

language mixing Switching languages within a response, a failure of R1-Zero when prompts were in one language and the training data in several. Fixed in R1 by a language-consistency reward.

cold start A small SFT stage before RL that teaches the format and style the RL will then optimize within, so that RL does not start from a base model's raw outputs.

AIME The American Invitational Mathematics Examination, 30 problems per year with integer answers. AIME 2024 and 2025 are the standard hard-math evaluations for reasoning models; accuracy is reported as pass@1 averaged over many samples.

majority voting Sampling several answers and returning the most common one (also consistency or cons@k). A cheap use of test-time compute that lifts accuracy without training.

language-consistency reward A reward proportional to the fraction of the reasoning written in the prompt's language, added in R1's stage 2.

Intuition

R1-Zero is a student given only an answer key and unlimited scratch paper. Nobody told them to check their work; the ones who did got more answers right, and so the habit spread. R1 is the same student given a few model solutions first so that their scratch work is legible, and then asked to write up their best solutions as a textbook for the next class.

5.3 Slide 17: Distillation, thinking budgets, and one model for both modes

Reasoning transfers by SFT on traces; a model can be taught to think only when asked, and to stop when told.

What the slide shows Top: a diagram of distillation. The teacher (R1, 671B) is sampled to produce 800k traces filtered for correctness; the student (Qwen2.5 or Llama 3) is trained on them by SFT only. A note: distillation beats RL for small models, and s1 showed 1,000 curated traces suffice. Middle: a token ribbon

showing budget forcing, with the model's </think> suppressed and "Wait" appended so that it continues thinking, and the answer following a later </think>. Right, a panel on Qwen3: four stages including thinking-mode fusion, and a thinking budget that caps thinking tokens and forces the answer.

Details Distillation here is the SFT sense of slide 5: sample the teacher, filter, fine-tune the student on the samples. R1's paper did this for six students (Qwen2.5-Math 1.5B and 7B, Qwen2.5 14B and 32B, Llama 3.1 8B and Llama 3.3 70B) using the same 800k samples as its stage 3, with SFT only and no RL on the students. The finding that matters is the comparison: a 32B Qwen trained by RL from scratch with R1-Zero's recipe reached AIME 2024 pass@1 of 47.0%, while the same base distilled from R1 reached 72.6%. The search for reasoning behavior is expensive and works best in a large model; once found, its traces are cheap to copy into a small one. This is why most small reasoning models since 2025 are distilled rather than RL-trained. s1 (Muennighoff et al. 2025) pushed the data to the minimum: 1,000 questions chosen for difficulty, diversity and quality, with reasoning traces from Gemini, fine-tuned into Qwen2.5-32B-Instruct in 26 minutes on 16 H100 GPUs, giving a model competitive with o1-preview on competition math.

Budget forcing is s1's second contribution and the ribbon illustrates it. The thinking length of a reasoning model is a dial that can be turned at inference by intervening on the end-of-thinking token: to make the model think longer, suppress </think> when it is generated and append "Wait", after which the model re-examines its work and often corrects it; to make it think less, insert </think> and force the answer. Accuracy rises with the forced length, which gives the test-time scaling curve of slide 15 for a single model without any search.

Qwen3 (2025) is the recipe for one model that can do both. Stage 1 is long-CoT cold start, as in R1; stage 2 is reasoning RL; stage 3 is thinking-mode fusion, an SFT on a mixture of thinking data (with reasoning in think tags) and non-thinking data (empty think tags), with /think and /no_think flags in the chat template (the report writes /no think; the model cards use the underscore) so that the user can choose; stage 4 is general RL over many tasks, including instruction following, format, agent tasks and preference. The result is one model that reasons when asked and answers directly otherwise, and because the thinking data were trained with explicit budgets, inserting a stop at any point and forcing the answer gives a quality that rises smoothly with the budget. Most 2025–26 open models ship this way: Qwen3, GLM, Kimi K2 and the DeepSeek V3.1 line all expose a thinking switch or budget.

Terms

teacher / student In distillation, the model whose outputs are learned from (teacher) and the model being trained (student). The student is usually smaller.

reasoning trace A response consisting of the thinking tokens and the final answer, used as an SFT example for distillation.

strong-to-weak distillation Qwen3's term for distilling from its large models into its small ones, first off-policy (SFT on teacher outputs) and then on-policy (slide 28).

s1 "Simple test-time scaling" (Muennighoff et al. 2025): 1,000 curated reasoning examples and budget forcing, enough to make Qwen2.5-32B-Instruct a competitive reasoner.

budget forcing Controlling thinking length at inference by suppressing or inserting the end-of-thinking token, appending "Wait" to extend.

thinking mode fusion Qwen3's stage 3: SFT on a mixture of thinking and non-thinking examples with mode flags, so that one model serves both.

thinking budget A cap on thinking tokens set by the user; when reached, the thinking is closed and the answer is generated. Quality rises with the budget.

non-thinking mode Answering directly, with empty think tags. Cheaper and faster for easy prompts.

Intuition

Finding a proof is hard; reading one is easy. A big model finds the proofs by trial and reward; a small model reads a few hundred thousand of them and learns the style of finding. And once a model thinks in writing, you can control how long it thinks the way you control a speaker: let them continue, or ask for the conclusion.

5.4 Slide 18: What reasoning RL does, and does not, add

Pass@1 rises a lot; pass@k at large k barely moves: RL mostly sharpens what the base model had.

What the slide shows A plot, labelled illustrative after Yue et al. (2025), of pass@k against k on a log axis from 1 to 1024, for a base model and the same model after RL. At $k = 1$ the RL model is far ahead (62% against 30%); as k grows the base model's curve rises faster and the two meet, or the base overtakes, by k in the hundreds. A panel reads it: sharpening, not discovery; overthinking; entropy collapse; evaluation.

Details Pass@k is the probability that at least one of k samples is correct. For a base model it rises steeply with k, because the base distribution is broad: a correct solution is in there at low probability, and enough samples will find it. For the RL model it starts high and rises slowly, because the distribution has been concentrated on a few solutions. Yue et al. (2025) measured this for several model families and RL recipes on math, code and visual reasoning, and found that at large k the base model matched or exceeded the RL model: every problem the RL model could solve, the base model could also solve given enough samples. Their reading is that RL, with current recipes, moves probability mass onto correct solutions that the base model could already produce; it sharpens rather than discovers. The same paper found that distillation from a stronger teacher does add solutions the base model lacked, which is consistent with slide 17.

The result is a statement about current recipes and sample budgets, and the slide says so: whether longer RL on harder problems, with more exploration, changes it is open, and some 2025–26 reports claim otherwise. It is also a useful piece of engineering guidance. If RL sharpens, then the base model's pass@k at large k is an upper bound on what RL can reach quickly, and a prompt on which the base model never succeeds in k samples is one on which RL will get no signal (all G wrong, zero advantage): choose RL prompts for which the base model succeeds sometimes.

The panel's remaining points are the three practical consequences. Overthinking: because longer responses have more chances to recover from an error, length is rewarded indirectly, and RL models think long even on easy questions; the fixes are length penalties (DAPO) and thinking budgets (slide 17). Entropy collapse (slide 14): sharpening is a fall in entropy, and if it goes too far the model cannot explore on the next task; clip-higher and entropy bonuses hold it open. Evaluation: pass@1 on a 30-question benchmark from a single sample is a number with a confidence interval of many points; reasoning models are evaluated by pass@1 averaged over many samples (avg@32 or avg@64 on AIME), and the protocol is reported with the number.

Terms

pass@k The probability that at least one of k independent samples solves the problem, estimated without bias from $n \geq k$ samples by $1 - \binom{n-c}{k} / \binom{n}{k}$ with c correct. Pass@1 is ordinary accuracy.

sharpening Concentrating the policy's probability on solutions the base model could already produce, as opposed to adding solutions it could not. Yue et al.'s characterization of current reasoning RL.

exploration Sampling responses the policy currently assigns low probability, which is how RL can find new behavior. Falls as entropy falls.

overthinking Producing long reasoning on problems that do not need it, a side effect of length being rewarded indirectly.

entropy bonus A term added to the RL objective that rewards higher per-token entropy, to maintain exploration.

avg@n Pass@1 estimated as the average accuracy over n samples per problem; the standard way to report AIME results for reasoning models.

confidence interval The range within which a benchmark score would vary over re-sampling. On a 30-question benchmark with one sample, it is about ± 15 points.

Intuition

A base model is a student who knows the answer but says it only one time in twenty; the RL model says it nineteen times in twenty. On the first attempt, the second student looks far better. Over twenty attempts, they look the same, which tells you the course taught confidence and selection, not new mathematics. The next class has to decide whether that is what it wanted, and most of the time it is.

6 Long context (slides 19–21)

6.1 Slide 19: Context: why a long window is its own stage

Models are pretrained at 4–8k tokens for cost; the window is stretched late, with a small fraction of the data.

What the slide shows A plot, labelled illustrative, of loss against token position on a log axis from 1k to 300k. A model trained to 8k has flat loss up to 8k and then climbs steeply. The same model with its positions stretched but no training is nearly flat to 128k with a small rise, then climbs. After an extension stage the curve is flat to 128k. Dotted lines mark the pretraining length and the target. A panel answers why not pretrain long from the start, lists the three things that change (position encoding, data, systems), and says where the stage sits in the pipeline.

Details The plot is the problem statement. A transformer with rotary position embeddings (RoPE, slide 20) encodes the distance between a query and a key as a rotation angle, and the angles it has learned to interpret are those that occurred during training. Past the trained length, the angles for the largest distances are ones it has never seen: attention spreads over the wrong tokens and the loss at those positions rises within a few thousand tokens of the boundary. Stretching the positions so that the new, longer range maps onto angles the model has seen (the dashed curve) removes most of the damage with no training at all, and a short training stage at the new length removes the rest.

Why not train long from the start? Two costs. Attention computes a score for every pair of tokens in the window, so its cost per layer grows with the square of the length: going from 8k to 128k multiplies the attention FLOPs by 256 and the activation memory accordingly, for every one of the trillions of pretraining tokens. And documents longer than 8k tokens are a small fraction of any corpus, so most of a long window would be filled with concatenated short documents that have nothing to say to each other. Pretraining therefore runs at 4–8k, and the long window is bought at the end with a small fraction of the data: Llama 3 used about 800B tokens (slide 21) against 15T of pretraining, and YaRN reports under 0.1% of the pretraining data.

Three things change in the extension stage. The position encoding is rescaled, by one of the methods of slide 20. The data are changed: long documents (books, code repositories, long articles) are up-sampled, and synthetic tasks over long inputs (questions whose answer requires facts from across a long document, summaries of long texts) are added, because natural long documents rarely require the model to use information far away. The systems change: training at 128k needs sequence parallelism (splitting one sequence across GPUs), FlashAttention (computing attention without materializing the score matrix), and chunked prefill at inference.

Where the stage sits is the last point. Llama 3 ran context extension on the 405B base model as the final stage of pretraining, before any SFT; most recipes do the same, treating it as mid-training. The later stages must then preserve the ability: an SFT mix of short conversations would let the model forget how to use a long window, so the SFT mix includes long examples (Llama 3: 0.1% synthetic long-context SFT data), and the long-context evaluations are re-run after every stage.

Terms

context window The maximum number of tokens the model can attend over in one forward pass; the length of prompt plus response it can handle.

context extension A training stage that raises the context window from the pretraining length to a longer target, by rescaling positions and training on long data.

position encoding The mechanism by which a transformer knows the order of tokens. RoPE encodes relative position as a rotation of query and key vectors.

RoPE Rotary Position Embedding (Su et al. 2021): each pair of dimensions in a query or key is rotated

by an angle proportional to the token's position, so that the dot product depends on the relative distance.

extrapolation Using a model at positions beyond those it was trained on. With RoPE, attention degrades quickly past the trained length unless the positions are rescaled.

sequence parallelism Splitting one long sequence across several GPUs so that its activations fit in memory, with communication for attention across the split.

FlashAttention An attention kernel that computes the softmax in blocks without materializing the full score matrix, making long-sequence attention memory-linear (Dao et al. 2022).

chunked prefill Processing a long prompt in chunks at inference so that memory stays bounded and decoding requests can be interleaved.

up-sampling Increasing the frequency of a data type (here long documents) in the training mix relative to its natural frequency.

Intuition

A model trained at 8k has only ever measured distances with an 8k ruler. Hand it a 128k document and every distance past the ruler's end reads as nonsense. Stretching the positions is re-marking the ruler so that 128k fits on it; the training stage is practice reading the finer marks. It is cheap because the model already knows how to read; it only needs the new scale.

6.2 Slide 20: Stretching RoPE: interpolation, base change, YaRN

Every method rescales the rotation frequencies so that unseen positions map onto seen angles.

What the slide shows Top left, one sentence of RoPE: pair i at position m is rotated by $m\theta_i$ with $\theta_i = b^{-2i/d}$; pair 0 turns once every few tokens, the last pair once in about b tokens; extending $L \to sL$ means choosing a new θ_i' per pair. Below it, three rows of twelve bars, one row per method, showing how much each dimension pair's positions are stretched for $s = 4$: PI stretches every pair by 4; NTK-aware stretches from 1 at the highest frequency to 4 at the lowest; YaRN leaves the high-frequency pairs at 1, ramps the middle, and stretches the low-frequency pairs by 4. Right, a table of five methods (PI, NTK-aware, YaRN, ABF, LongRoPE) with what changes, the training needed and the reach, and a panel on what open models do now.

Details RoPE assigns the i-th pair of dimensions, $i = 0, \ldots, d/2-1$, the angular frequency $\theta_i = b^{-2i/d}$ with base $b = 10{,}000$ in the original. A token at position m has its pair rotated by $m\theta_i$, and the attention score between positions m and n depends on $(m-n)\theta_i$. The highest-frequency pair ($i = 0$, $\theta_0 = 1$ radian per token) completes a turn every 2π tokens and encodes local order; the lowest ($i = d/2-1$, $\theta \approx 1/b$) completes a turn every $2\pi b \approx 63{,}000$ tokens and encodes coarse position. Extension by a factor s means the model must handle distances up to sL, and the question is how to change the θ_i so that the angles seen at those distances are ones it learned.

Position interpolation (PI, Chen et al. 2023) divides every position by s, equivalent to $\theta_i' = \theta_i/s$ for all i: every pair is stretched by the same factor, as the first row of bars shows. All angles then lie within the trained range, and about a thousand fine-tuning steps at the new length restore performance; the cost is that the high-frequency pairs, which resolve nearby tokens, are made s times coarser, so local resolution suffers. NTK-aware scaling (bloc97, 2023, a Reddit post, hence the †) instead raises the base to $b' = b\,s^{d/(d-2)}$, which leaves the highest frequency unchanged and stretches the lowest by exactly s, with the pairs between stretched by increasing factors (the second row); it works without any training for small s. ABF (adjusted base frequency, Xiong et al. 2023) is the same move with a large fixed base (5×10^5) and continued pretraining at

the new length; it is what Llama 2 Long and Llama 3 did. YaRN (Peng et al. 2023) makes the stretch depend on each pair's wavelength: pairs whose wavelength is shorter than the trained length by a margin are not stretched at all (they encode local order and were trained on every angle already), pairs whose wavelength exceeds the trained length are stretched by s (they were never trained on a full turn), and pairs between are ramped. The third row shows this. YaRN also multiplies the attention logits by $1/\sqrt{t}$ with $\sqrt{1/t} = 0.1 \ln s + 1$, a temperature that counteracts the flattening of attention over a longer window; with under 0.1% of the pretraining data it reaches 128k. LongRoPE (Ding et al. 2024) searches per-pair factors by an evolutionary algorithm against perplexity, fine-tunes at 256k, and reports a reach of 2048k.

What open models do now, from the panel: raise the base early, during pretraining, so that the model trains with coarser low frequencies from the start (Llama 3 uses 5×10^5, Qwen2.5 and Qwen3 10^6); train the long stage at 32k–128k; and apply YaRN factors (with dual chunk attention, in Qwen3's case) at inference for a further $4\times$ when a longer window is needed, which Qwen3 documents as a supported configuration. DeepSeek-V3 extended in two YaRN phases, 4k to 32k and 32k to 128k.

Terms

rotation frequency $\theta_i = b^{-2i/d}$, the angle per token by which dimension pair i is rotated. High for low i, low for high i.

RoPE base b, the constant that sets the spread of frequencies; 10,000 originally, 5×10^5 to 10^6 in recent models to allow longer windows.

wavelength $2\pi/\theta_i$, the number of tokens over which pair i completes one rotation. Pairs with wavelength longer than the trained length never saw a full turn.

position interpolation (PI) Dividing all positions by the extension factor s (Chen et al. 2023), so that the extended range maps into the trained range. Needs fine-tuning.

NTK-aware scaling Raising the RoPE base by $s^{d/(d-2)}$ so that high frequencies are kept and low frequencies are stretched (bloc97 2023). Named for a neural-tangent-kernel argument; works without training for small s.

ABF Adjusted base frequency (Xiong et al. 2023): a large fixed RoPE base with continued pretraining at the longer length.

YaRN "Yet another RoPE extensioN" (Peng et al. 2023): by-parts interpolation of frequencies according to wavelength, plus an attention temperature, with a small amount of fine-tuning.

attention temperature A scalar multiplying the attention logits before the softmax. YaRN lowers it slightly for long windows to keep attention from flattening.

LongRoPE Ding et al. 2024: per-dimension rescaling factors found by evolutionary search, with progressive fine-tuning, reaching a 2M-token window.

Intuition

RoPE is a set of clocks running at different speeds, from a second hand to a hand that takes a day to go round. Extending the window is making the day longer. PI slows every clock equally, so the second hand becomes useless. NTK and YaRN slow only the slow clocks, which were the ones that would have run off the dial, and leave the fast ones alone. The training stage is learning to read the slowed clocks.

6.3 Slide 21: The long-context protocol: stages, data, and checks

Extend in steps, gate each step on recovery of short-context scores and a perfect needle test.

What the slide shows Left, a staircase of six stages from 8k to 128k, with the note that Llama 3 405B used about 800B tokens in total over six stages and that the intermediate lengths are not published; a brace beneath states the gate between stages. Middle, a plot labelled illustrative after Liu et al. (2023): accuracy against the position of the relevant document among twenty, a U shape, high at the first and last positions and low in the middle. Right, a panel on checks: NIAH, RULER, and re-running the short suite. Below, a note on data.

Details The protocol is Llama 3's (Grattafiori et al. 2024, Section 3.4.2) and is representative. Extension is done in increments rather than one jump, because each increment is a smaller change in the attention statistics and recovers with less data; six stages from 8k to 128k, with about 800B tokens in total for the 405B model. Between stages there are two gates. Short-context evaluations must return to their values before the stage: the usual damage from context extension is a drop on ordinary benchmarks, as the model's attention is spread over the longer window, and it must be recovered before the next stage. And the needle-in-a-haystack test must be solved perfectly at the new length: a sentence is hidden at a random depth in a long filler document and the model is asked for it; anything short of perfect means the model cannot yet use the new positions.

The middle plot is the reason perplexity is not the evaluation. Liu et al. (2023) gave models a question and 10, 20 or 30 retrieved documents, one of which contained the answer, and moved that document through the positions. Accuracy was high when the relevant document was first or last and fell by more than 20% when it was in the middle (the plot is the 20-document case); in the worst case it fell below the closed-book accuracy with no documents at all, for models whose context window comfortably held them. A model can have low perplexity on long text and still not use information in the middle of its window. Evaluations must therefore be position-aware: the needle test varies the depth, and RULER varies the task.

RULER (Hsieh et al. 2024) is the standard suite: 13 tasks in four categories, retrieval (needles of several kinds, multiple needles, with distractors), multi-hop tracing (follow a chain of variable assignments through the document), aggregation (count frequent words across the document), and question answering over long inputs, each generated synthetically at any length. Its finding was that most models that claimed a 32k or 128k window performed well only to a fraction of it; the effective length is where the score falls below a threshold, and it is usually shorter than the claimed one.

The data note closes the loop with slide 19. The extension stage's data are long documents up-sampled plus synthetic long tasks. The SFT mix afterwards must also contain long examples, or the ability is lost; Llama 3 generated synthetic long-context SFT data (QA, summarization and code reasoning over long documents) and found 0.1% of the mix sufficient. DPO on short preference data alone did not hurt long-context performance in their ablation, which is the kind of result that only an ablation can give.

Terms

needle in a haystack (NIAH) A test (Kamradt 2023) that hides a sentence at a chosen depth in long filler text and asks the model to retrieve it, scored over a grid of lengths and depths.

lost in the middle Liu et al.'s 2023 finding that models use information at the start and end of a long context more reliably than information in the middle.

RULER A synthetic long-context benchmark (Hsieh et al. 2024) with 13 tasks in four categories, generated at any length; its effective-length measure exposes windows that are claimed but not usable.

effective context length The longest input at which a model keeps a benchmark score above a threshold (RULER uses the Llama 2 7B score at 4k). Usually shorter than the claimed window.

gate A criterion that must be met before the next stage of a protocol runs: recovered short-context scores and a perfect needle test, in the long-context protocol.

short-context regression The drop on ordinary benchmarks that context extension causes, which the gate checks for.

synthetic long-context data Training examples constructed by placing questions, summaries or code tasks over long documents, used because natural long documents rarely require long-range use.

Intuition

Giving a model a 128k window is like giving a reader a longer desk: the test is not whether the papers fit on it but whether the reader can find the one that matters when it is in the middle of the pile, and whether they can still do short tasks as well as before. The protocol widens the desk a little at a time and does not widen it again until both are true.

7 Safety (slides 22–24)

7.1 Slide 22: Safety alignment: targets and data

Helpful, honest, harmless. The stage tunes one trade-off and needs data on both sides of the line.

What the slide shows Left, a plot labelled illustrative: violations on unsafe prompts against false refusals on safe prompts, a curve falling from a "permissive" point (few false refusals, many violations) to an "over-refusing" point (the reverse), with a target between. Below it, HHH and the note that Llama 2 and 3 report both rates and tune the data mix until both are acceptable; XSTest (250 safe and 200 unsafe prompts) measures the refusal side. Middle, the Constitutional AI loop: a red-team prompt and first response, critique against a written principle, revision, SFT on the revisions, repeated per principle; then RL with AI preference labels. Right, a panel on where the data come from: human red teams and a safety reward model, model critiques, borderline prompts.

Details The targets are Askell et al.'s (2021) three: helpful, honest, harmless. The slide's plot is the tension between the first and the third. A model that never refuses violates on every harmful request; a model that refuses anything that sounds dangerous also refuses "how do I kill a Python process" and the many safe prompts that share surface features with unsafe ones. Safety training moves the model along the curve, and the only way to know where it is on the curve is to measure both rates. Llama 2 (Touvron et al. 2023) introduced the practice of reporting a violation rate on adversarial prompts and a false-refusal rate on safe ones, and Llama 3 tuned its safety data mix, by category, until both were acceptable. XSTest (Röttger et al. 2023) is the benchmark for the refusal side: 250 safe prompts constructed to look unsafe (homonyms, figurative language, safe targets, historical events) and 200 contrasting unsafe ones; a well-tuned model should comply with the first set and refuse the second.

The middle of the slide is Constitutional AI (Bai et al. 2022), which replaces most human harmlessness labels with the model's own. In the supervised phase, the model is given a red-team prompt and produces a response (which, from a helpful-only model, is often harmful); it is then asked to critique the response against one principle drawn from a written list (the constitution), and to revise it accordingly; the critique and revision are repeated with other principles; and the model is fine-tuned on the final revisions. In the RL phase, the model's responses to red-team prompts are compared in pairs by the model itself against a principle, the resulting AI preference labels train a reward model, and RL proceeds as in RLHF. The helpfulness labels remain human. The result in the paper was a model that was less harmful than the human-labelled RLHF baseline and also less evasive: it explained its refusals rather than simply refusing, since the revisions were trained to do so.

The right panel is the data inventory. Human red teams write attacks, and annotators choose the safer of two responses, giving pairs for a reward model; Llama 2 trained a separate safety reward model alongside its helpfulness one, and used context distillation, in which the model answers under a safety-oriented system prompt and is then trained to give that answer without the prompt. Model critiques, as in CAI, replace most human harmlessness labelling. And borderline prompts with safe completions, so that the model learns that refusing is not the only safe answer: a safe completion to a borderline medical or security question is often better than a refusal, and a dataset containing only refusals teaches over-refusal.

Terms

HHH Helpful, honest, harmless (Askell et al. 2021), the three targets of an aligned assistant.

violation rate The fraction of unsafe prompts to which the model gives a harmful response.

false refusal rate The fraction of safe prompts the model refuses. Also called over-refusal.

XSTest A test of over-refusal (Röttger et al. 2023): 250 safe prompts that superficially resemble unsafe

ones and 200 unsafe contrasts.

red teaming Deliberately attacking a model with prompts designed to elicit harmful output, to find failures and to produce safety training data.

Constitutional AI (CAI) Bai et al. 2022: harmlessness training in which the model critiques and revises its own responses against written principles (supervised phase) and labels its own comparisons (RL phase).

constitution The written list of principles the model critiques against in CAI, e.g. "choose the response that is least harmful".

critique and revision The CAI loop step in which the model writes a criticism of a response against a principle and then a revised response.

safety reward model A reward model trained on safety comparisons only, used by Llama 2 alongside a helpfulness reward model.

context distillation Training a model to produce, without a system prompt, the responses it gives with one. Used by Llama 2 to internalize safety instructions.

borderline prompt A prompt near the boundary between safe and unsafe, for which the best response is a careful answer rather than a refusal or full compliance.

Intuition

Safety training is tuning a single dial between "answers everything" and "refuses everything", and the dial can only be set by looking at both ends at once. The data have to show the model what to refuse and what not to refuse; a training set of refusals alone just turns the dial all the way.

7.2 Slide 23: Measuring safety: red teams, jailbreaks and guards

The aligned weights are one layer. Optimized attacks transfer across models; classifiers and specs add layers.

What the slide shows Top, a layered pipeline: prompt, input guard (a classifier), the model (weights and system prompt), output guard, answer. Four attacks beneath it point at the layer they target: hand-written jailbreaks (role play, "ignore previous instructions"); GCG, a suffix optimized by gradient on an open model and pasted into the prompt; many-shot, hundreds of faux dialogues placed in the context; and fine-tuning the weights on a few examples (next slide). Bottom left, GCG transfer results from Zou et al. (2023), Table 2, "ensemble": attack success rates of suffixes optimized on Vicuna and Guanaco against GPT-3.5 86.6%, GPT-4 46.9%, Claude 1 47.9%, Claude 2 2.1%, PaLM-2 66.0%, over 388 behaviors. Right, a panel on benchmarks and tools: HarmBench, XSTest, Llama Guard, deliberative alignment.

Details The slide's claim is that the aligned weights are one layer of defence, not the defence. A jailbreak is any input that gets an aligned model to produce output it was trained to refuse. The hand-written kind (role play, instructions to ignore prior instructions, encoding the request) works by moving the prompt away from the distribution of the safety training data. GCG (Greedy Coordinate Gradient, Zou et al. 2023) automates this: it appends a suffix of nonsense tokens to a harmful request and optimizes the tokens, by gradient on an open-weight model, to maximize the probability that the response begins "Sure, here is". The suffixes are meaningless to a reader and very effective on the model they were optimized on. The transfer result is the alarming part: suffixes optimized on Vicuna and Guanaco, both small open models, and tested on closed models for which no gradients were available, succeeded 86.6% of the time on GPT-3.5 and 46.9% on GPT-4 in the "ensemble" setting, where a behavior counts as attacked if any of the optimized suffixes works. Claude

2 at 2.1% is the outlier. One open model's gradients attack the others, because the models share training data and the safety training shares its shape. Many-shot jailbreaking (Anil et al. 2024, NeurIPS; the deck's † refers to Anthropic's research write-up of it) fills a long context with hundreds of fabricated dialogues in which an assistant complies, and the in-context learning that makes long windows useful makes the model comply too; success rises as a power law in the number of shots. The fourth attack, fine-tuning, is the next slide.

The layers are the response. An input classifier can reject a prompt that the model would have answered; an output classifier can block a harmful completion; a system prompt can state the policy. Llama Guard (Inan et al. 2023) is an LLM fine-tuned as a classifier of both inputs and outputs against a taxonomy of hazards, released alongside the Llama weights so that deployers have the layer; version 3 covers 14 hazard categories in the 8B model, 13 in the 1B (†: from the model cards). HarmBench (Mazeika et al. 2024) made attacks and defences comparable: 510 harmful behaviors, 18 attack methods including GCG and its variants, 33 target models, and one scoring protocol with a classifier for whether the output is actually harmful; before it, every attack paper used its own behaviors and its own judge. XSTest measures the other side, so that a defence that works by refusing more is caught. Deliberative alignment (Guan et al. 2024) is the reasoning-model version of safety training: the model is given the written safety policy, trained by SFT on chains of thought that cite it, and then by RL with a judge that holds the policy; the model reasons about the policy at inference rather than pattern-matching the prompt, which improved both jailbreak resistance and over-refusal in the paper's results.

Terms

jailbreak An input that causes an aligned model to produce output it was trained to refuse.

GCG Greedy Coordinate Gradient (Zou et al. 2023): an optimized adversarial suffix found by gradient-guided token search on an open model, which transfers to other models.

attack success rate (ASR) The fraction of harmful behaviors for which an attack elicits a harmful response, as judged by a classifier or a person.

transfer attack An attack optimized on one model that also works on another, including closed models whose gradients are unavailable.

many-shot jailbreaking Filling a long context with fabricated compliant dialogues so that in-context learning overrides the safety training (Anil et al. 2024).

input guard / output guard Classifiers that screen the prompt before the model sees it and the response before the user does.

Llama Guard An LLM-based input and output safety classifier (Inan et al. 2023) released with Llama, trained against a hazard taxonomy; version 3 covers 14 categories.

HarmBench A standardized evaluation of automated red teaming (Mazeika et al. 2024): 510 behaviors, 18 attacks, 33 models, one judge.

deliberative alignment Guan et al. 2024: training a reasoning model to read and reason over the written safety policy, by SFT on policy-citing chains of thought and RL with a policy-aware judge.

safety policy / spec A written document stating what the model should and should not do, used as a training target and as a reference for judges.

Intuition

Safety training teaches the model to say no; a jailbreak is a way of asking that the training did not anticipate, and an optimized jailbreak is a way of asking that no human would think of. Since the question can always be rephrased, the weights cannot be the only lock. The guards are a second lock on the door

and a third on the window, and the benchmarks are the inspection that checks all of them at once.

7.3 Slide 24: Open weights: fine-tuning undoes alignment

A few examples or a benign dataset erode safety; the alignment is only a few tokens deep.

What the slide shows Left, three bars labelled illustrative after Qi et al. (2023): harmful-answer rate as released (low), after fine-tuning on 10 adversarial examples (high), after benign fine-tuning (intermediate). Middle, a plot labelled illustrative after Qi et al. (2024): the KL divergence between the aligned and the base model at each output token position, large at the first few tokens and near zero after. Below, the two papers' findings. Right, a panel on what open releases do: deepen the alignment, ship the other layers, remove hazardous knowledge, decide what to release.

Details The slide is about a vulnerability that only open weights have, and that no closed model's defences address: anyone with the weights can continue training them. Qi et al. (2023) showed how little it takes. Fine-tuning GPT-3.5 Turbo through OpenAI's API on 10 examples of harmful instructions with compliant answers, at a cost under $0.20, removed its refusals almost entirely; so did fine-tuning on a small set of examples that merely trained the model to be absolutely obedient; and, most relevant to practice, fine-tuning on a benign dataset (Alpaca) with no harmful content also degraded safety measurably, because the safety behavior is a small part of the weights and any training moves it. The same results held on open Llama 2. The lesson is that alignment is not a property that survives fine-tuning, and a release that assumes it will is mistaken.

Qi et al. (2024) explain why it is so fragile. Comparing the aligned model with its base model token by token, they found that the KL divergence between them is large at the first few output tokens and near zero after: the alignment consists mostly of a learned preference for beginning the response with a refusal ("I cannot help with that") rather than with "Sure". Once the first tokens are fixed, by a prefix in the prompt, by a jailbreak that elicits "Sure", or by a few fine-tuning examples, the rest of the response is generated by what is effectively the base model. They call this shallow safety alignment and propose two fixes: training on examples in which the response begins with a harmful prefix and recovers (so that the alignment is present deeper in the response), and a constrained fine-tuning objective that penalizes changes to the first tokens' distribution, which makes benign fine-tuning safer for a deployer who exposes it.

The panel is what open releases can do, and the slide's last line is that none of it is settled. Deepen the alignment with the recovery data and the constrained objective. Ship the other layers: Llama Guard and prompt classifiers released with the weights, so that a deployer has defences that do not live in the fine-tunable weights. Remove hazardous knowledge rather than teaching refusal: WMDP (Li et al. 2024) is a benchmark of hazardous knowledge in biosecurity, cybersecurity and chemistry, and RMU is an unlearning method that scrambles the model's internal representations on that knowledge while keeping general ability; tamper-resistant training (TAR, Tamirisa et al. 2024) trains the weights so that fine-tuning attacks fail to recover the removed capability, and holds for hundreds of adversarial fine-tuning steps, not indefinitely. And decide what to release: an uplift evaluation, before the weights go out, of whether the model gives a capable adversary more than they could already get, with the release decision made on the result.

Terms

fine-tuning attack Removing a model's safety behavior by continuing to train it, on a few harmful examples or even on benign data.

shallow safety alignment Qi et al.'s 2024 finding that the aligned and base models differ mainly in the first few output tokens, so that the alignment can be bypassed by controlling those tokens.

prefix attack Forcing the first tokens of the response (e.g. "Sure, here is") so that the rest is generated

as by the base model.

recovery data Training examples in which a response begins with a harmful prefix and then refuses or redirects, used to make the alignment present at later positions.

constrained fine-tuning A fine-tuning objective that penalizes changes to the distribution of the first output tokens, so that benign fine-tuning does not erode safety.

unlearning Training a model to lose specific knowledge or capability while keeping the rest. RMU (Li et al. 2024) does this by steering internal representations on the target data.

WMDP Weapons of Mass Destruction Proxy (Li et al. 2024): a benchmark of 3,668 questions proxying hazardous knowledge in biology, chemistry and cyber, used to measure unlearning.

RMU Representation Misdirection for Unlearning: the unlearning method of the WMDP paper, which pushes the model's activations on hazardous inputs toward a random direction.

tamper-resistant training (TAR) Tamirisa et al. 2024: training the weights so that subsequent fine-tuning attacks fail to recover a removed capability, by meta-learning against simulated attacks.

uplift evaluation An assessment of how much a model increases a capable adversary's ability to cause harm beyond what is already available, used to inform release decisions.

Intuition

The refusal lives in the first few words. A model that has learned to begin with "I can't" has learned almost nothing else about the request, so anything that changes the first few words (a prefix, a jailbreak, ten fine-tuning examples) changes everything. For an open model, whose owner can retrain the first words at will, safety has to be something other than the weights' first instinct: knowledge that is not there, defences that are not in the weights, and a decision about what to release at all.

8 Tool use (slides 25–26)

8.1 Slide 25: Tool use I: function calling is a format, taught by SFT

The model emits a structured call between special tokens; the runtime executes it and appends the result.

What the slide shows A five-turn transcript. System: the chat template inserts the tool schemas (`get_weather` with a `city` parameter). User: what is the weather in Chicago today? Assistant: a JSON call to `get_weather` with `city` Chicago, between `<tool_call>` tags. Tool: the runtime ran it and returned a forecast of rain with a high of 54. Assistant: rain in Chicago today, with a high of 54 °F. Each turn is tagged "masked" or "loss": only the two assistant turns carry loss. A panel on how it is taught: the data (APIGen, ToolBench, negatives), evaluation (BFCL, τ-bench), failures.

Details Function calling is nothing but a format. The deployer declares the available functions as schemas (name, description, typed parameters, usually JSON Schema); the chat template renders them into the system turn; the model, when it decides a function is needed, emits a call as structured text between special tokens instead of a natural-language answer; the runtime parses the call, executes the function, and appends the result as a new turn with a tool role; and the model continues, now with the result in its context. The special tokens and the exact JSON layout differ between model families (Llama, Qwen, Mistral and Hermes each have their own), which is why a model must be served with its own template. The assistant's final sentence is ordinary generation conditioned on the tool turn.

Because it is a format, it is taught the way formats are taught: by SFT on transcripts of this shape, with the loss on the assistant's tokens only. The two assistant turns in the example carry the loss; the system, user and tool turns are masked, and masking the tool turn is essential, since otherwise the model learns to write plausible tool results instead of waiting for them. The data are almost entirely synthetic. APIGen (Liu et al. 2024) generated 60k examples over 3,673 executable APIs, each verified in three stages (the call parses, it executes, and the result is judged consistent with the query), which is the kind of execution-checked synthesis that most tool data now uses. ToolBench (Qin et al. 2023) collected 16,464 real REST APIs and generated multi-step instructions over them, with ChatGPT producing solution paths by a depth-first search over calls. The data must also contain negatives: queries for which no tool is appropriate and the model should simply answer, and queries for which the needed tool is absent and the model should say so; without them the model calls a tool whenever one is declared.

Evaluation is by execution or by exact match on the call. The Berkeley Function Calling Leaderboard (BFCL, Patil et al. 2025) checks single calls, parallel calls (several in one turn), multiple-function selection, and, in its later versions, multi-turn conversations in which the model must track state across calls; scoring is by abstract-syntax-tree comparison of the call or by execution. τ-bench (Yao et al. 2024) evaluates the model as an agent in a simulated domain (retail, airline) talking to a simulated user and calling APIs over a database, with the final database state checked against the goal; its pass^k metric requires the same task to succeed in all k of k trials, which exposes unreliability that a single pass rate hides. The failures the panel lists are the ones these catch: invented functions, wrong argument types, a call when none was needed, and a final answer that ignores the result.

Terms

function calling The protocol by which a model requests execution of a declared function by emitting a structured call, which the runtime executes and returns.

tool schema The declaration of a function's name, description and parameter types, rendered into the system turn by the chat template.

tool turn A conversation turn with the tool role, containing a function's result, appended by the runtime

after it executes a call.

runtime The serving-side code that parses calls, executes functions and appends results; the model never executes anything itself.

APIGen Liu et al. 2024 (Salesforce): a pipeline for generating function-calling data verified by format, execution and semantic checks; 60k examples over 3,673 APIs were released.

ToolBench Qin et al. 2023: an instruction set over 16,464 real APIs from RapidAPI with multi-step solution paths, and the ToolLLaMA models trained on it.

negative example A training example in which the correct behavior is not to call a tool, or to report that no suitable tool exists.

BFCL Berkeley Function Calling Leaderboard (Patil et al. 2025): an evaluation of single, parallel, multiple and multi-turn function calls by AST match and execution.

τ-bench Yao et al. 2024: an agent benchmark with simulated users and tool APIs over a database in retail and airline domains, scored on the final state, with the pass^k reliability metric.

pass^k The probability that all k of k independent trials of the same task succeed; falls steeply with k for an unreliable agent, unlike pass@k.

Intuition

Teaching a model to call a tool is teaching it a new sentence shape: "I would like to look that up", in a grammar a program can parse. SFT is enough for the grammar. What SFT cannot teach is judgment about when to speak that sentence, which is why the data need examples of not calling, and why the next slide uses RL.

8.2 Slide 26: Tool use II: agentic RL over many turns

The same policy gradient, but the trajectory is a long dialogue with an environment and one reward at the end.

What the slide shows A trajectory as a chain of chips: task, think and call, tool output, think and call, tool output, …, final answer, then a reward box: tests pass, 1 or 0. A note says that the loss and advantage apply to the model's tokens only, that tool outputs are masked, and that one scalar at the end is spread over every model turn, with one advantage per trajectory in GRPO or one per turn in some recipes. Left, what changes against single-turn RLVR; right, where it is done: SWE-bench Verified, Kimi K2, tooling.

Details Agentic RL is the RL of Section 3 applied to a trajectory rather than a response. The policy produces a turn (reasoning and a tool call), the environment responds (a tool output), the policy produces another turn, and so on until it produces a final answer or hits a turn limit; the whole sequence of model turns is one sample, and it receives one reward at the end, from a verifier: the hidden tests pass, the user's goal state is reached, the answer matches. The objective is GRPO's, with the loss and the importance ratio computed over the model's tokens only and the tool outputs treated as context (masked, as in SFT). The single final reward gives one advantage per trajectory, applied to every model token in it; some recipes instead compute an advantage per turn, from a value estimate or a turn-level reward, to give some credit assignment across turns.

Four things change against single-turn RLVR, and the slide lists them. Trajectories are long: tens to a hundred turns, with tool outputs (file listings, test logs, web pages) making up most of the tokens, so the context fills and generation is slow; and because each turn blocks on a tool call, synchronous rollout leaves the

engine idle, so generation must be asynchronous across many trajectories. Environments must be sandboxed (the policy runs code), resettable (every rollout starts from the same state), and fast (thousands of rollouts per step); a container per rollout for coding tasks, a simulator for users or browsers. Rewards are sparse and final, and the behavior the policy must learn includes recovery from tool errors, which only appears in long trajectories and is where much of the gain lies. And credit assignment is unsolved: of forty calls in a successful trajectory, which mattered? The single-advantage recipe says all of them equally, which is what GRPO can do, and it works well enough that it is what most 2025–26 agentic RL uses.

Where it is done. SWE-bench Verified (†, OpenAI, August 2024) is a 500-task subset of SWE-bench (Jimenez et al. 2023), human-validated to remove tasks with underspecified issues or broken tests; each task is a real GitHub issue in a Python repository, the agent must produce a patch, and the reward is whether the hidden tests pass. It is the standard environment and the standard score for coding agents. Kimi K2 (Moonshot, July 2025) described a large synthetic agentic pipeline: thousands of tools, real and generated, simulated users and environments, trajectories filtered by quality for SFT, then RL with verifiable rewards where available and a self-critique rubric reward elsewhere. The tooling is the RL frameworks of slide 14 with agent loops added (veRL, slime), environment collections (NVIDIA's NeMo Gym, †), and OpenRLHF; in every case the rollout engine, not the trainer, is the bottleneck.

Terms

agent A model used in a loop that takes actions (tool calls) in an environment, observes results, and continues until a task is done.

trajectory The full sequence of model turns and environment responses for one task, from the initial prompt to the final answer. The unit of agentic RL.

environment The system the agent acts on: a code repository in a container, a simulated user, a browser, a database. Must be sandboxed and resettable for RL.

turn-level advantage An advantage computed per model turn rather than per trajectory, from a value estimate or turn-level reward; a partial answer to credit assignment.

SWE-bench Jimenez et al. 2023: real GitHub issues from Python repositories, solved by producing a patch that passes hidden tests. SWE-bench Verified is the 500-task human-validated subset.

Kimi K2 Moonshot's 1T-total, 32B-active open MoE (July 2025), post-trained with large-scale synthetic agentic data and RL with verifiable and self-critique rewards.

veRL / HybridFlow Sheng et al. 2024: an RL training framework that couples a training backend with an inference engine; the most used open framework for GRPO and agentic RL.

slime An RL framework from the GLM team built on SGLang rollouts and Megatron training, with agent-loop support.

NeMo Gym NVIDIA's collection of RL environments for agentic training (†: GitHub).

OpenRLHF An open RL framework built on Ray, vLLM and DeepSpeed, supporting PPO, GRPO and related algorithms.

Intuition

Single-turn RL grades an exam answer. Agentic RL grades a week of work on a project: the model makes dozens of decisions, sees the consequences of each, and gets one mark at the end. The gradient has no idea which decision earned the mark, so it credits all of them; that this works at all is because, averaged over many projects, the decisions that help are the ones that appear more often in the ones that succeed.

9 Recipes and evaluation (slides 27–30)

9.1 Slide 27: Open recipes, side by side

The shape is stable; the weight of each stage, and where the data comes from, moved between 2024 and 2026.

What the slide shows A table with one row per open recipe and columns for SFT, preference, RL, and the reasoning, context, safety and tools stages. Llama 3.1 405B (July 2024): rejection-sampled and synthetic SFT over six rounds, DPO each round, no online RL, long context in pretraining, tools by SFT, safety tuned on violation against false-refusal rates. Tülu 3 (November 2024): 939k-prompt SFT, length-normalized DPO, RLVR on GSM8K, MATH and IFEval, with all data and code open. DeepSeek-R1 (January 2025): cold start then rejection-sampling SFT, no DPO (reward models in the final RL), GRPO twice, R1-Zero from the base, distilled students. Qwen3 (April 2025): long-CoT cold start and thinking-mode fusion, no preference stage reported, reasoning RL then general RL, thinking budget, small models by distillation. Kimi K2 (July 2025): SFT with large synthetic agentic data, no preference stage reported, verifiable and self-critique rubric RL, MuonClip. OLMo 3 (November 2025, †): SFT, DPO, RLVR, in Think, Instruct and RL-Zero variants. 2026 releases (†): SFT, often no preference stage, staged RL as the centre, on-policy distillation from specialist teachers. A reading below.

Details Read the table column by column. The SFT column never empties: every recipe starts with demonstrations, and what changed is their source, from human and academic data to rejection-sampled and synthetic data checked by execution (Llama 3 and R1 both generate most of their SFT data from their own models). The preference column thins from top to bottom: DPO each round in Llama 3, one DPO stage in Tülu 3 and OLMo 3, none reported in R1, Qwen3 and Kimi K2, where the preference signal enters as a reward model inside the final RL. The RL column fills: none online in Llama 3, a short RLVR stage in Tülu 3, two GRPO stages in R1, two in Qwen3, and in the 2026 releases a staged RL that is the centre of the recipe and consumes most of the post-training compute. The last column shows the other three behaviors as their own data problems in every recipe, handled where the data are available: context in pretraining, tools by SFT and then by RL, safety as a tuned mix.

Three rows deserve a note. Tülu 3 is the row to read first as a student, because everything in it is open: the data mixes, the code, the evaluation suite and the ablations that justified each choice, including the finding that length-normalized DPO outperformed standard DPO on their mix, and that RLVR gave gains on the three targeted benchmarks without regressions. Llama 3 is the row that shows a frontier result without online RL at all, which was the state of the art in mid-2024 and is useful calibration for what rejection sampling and DPO can do with enough rounds. The two † rows are built from release notes and commentary rather than papers: OLMo 3's recipe is documented but was not yet in a peer-reviewed paper at the time of the deck, and the 2026 row (DeepSeek-V4, GLM-5, MiMo-V2-Flash, Nemotron 3) summarizes what the labs said about their own recipes. The one ablation in that row, Nemotron 3's finding that a preference stage still helped, is the one claim in it that a controlled experiment supports.

The reading at the bottom is the summary of the section. The sequence (SFT, then preference or RL, with reasoning RL from 2025) has been stable since InstructGPT. What moved is the weight on each stage, RL up and DPO down, and the source of data, toward synthetic data checked by execution. A recipe written in 2026 looks like one written in 2022 with the proportions changed.

Terms

MuonClip Kimi K2's optimizer: Muon (an orthogonalized momentum optimizer) with a clipping of attention logits to keep training stable at 1T parameters.

RL-Zero A variant trained by RL directly from the base model with no SFT, after DeepSeek's R1-Zero; OLMo 3 released one.

specialist teacher A model trained by RL on one domain (math, code, agents) whose behavior is distilled into a general student. The 2026 pattern for combining several RL runs.

staged RL Several RL stages in sequence, each on a different task mix or reward set, as in R1's two stages and Qwen3's reasoning-then-general RL.

release notes A lab's own description of a model at release, in a blog post or model card. Less detailed and less checked than a paper; claims from them are marked † in the deck.

Intuition

Every recipe in the table is the staircase of slide 1 with different step heights. The 2024 recipes climb mostly by SFT and DPO with a small RL step at the top; the 2026 ones climb mostly by RL with SFT as the landing. The staircase did not change shape because the signals of slide 3 did not change; the budget moved to the one that scales with compute.

9.2 Slide 28: On-policy distillation: dense signal, student's samples

Between SFT and RL: the student samples, the teacher scores every token. The 2026 consolidation step.

What the slide shows A two-by-two grid. Columns: off-policy (the teacher's samples) and on-policy (the student's samples). Rows: dense (a signal per token) and sparse (a signal per answer). Off-policy and dense: SFT on teacher traces, copies the teacher's tokens, off-distribution. On-policy and dense: on-policy distillation, student samples and the teacher scores every token. Off-policy and sparse: rejection-sampling SFT. On-policy and sparse: RLVR and GRPO, one reward per answer. Below, the per-token reward $r_t = \log \pi_T(y_t \mid x, y_{<t}) - \log \pi_S(y_t \mid x, y_{<t})$ and an explanation. Right, a panel on who uses it: GKD, Qwen3, Thinking Machines, multi-teacher OPD.

Details The grid organizes four methods by two questions: whose samples, and how much signal per sample. SFT on teacher traces (slide 17) has the densest possible signal, the teacher's exact token at every position, but on the teacher's distribution, so the student learns tokens in contexts it would not itself have reached (exposure bias). RLVR has the student's own samples but one bit of signal per answer, and must find good answers by search. Rejection-sampling SFT is in between on both counts. On-policy distillation takes the student's samples and gives them a dense signal: the teacher's log-probability of each token the student produced.

The per-token reward is

$$r_t = \log \pi_T(y_t \mid x, y_{<t}) - \log \pi_S(y_t \mid x, y_{<t}), \qquad y \sim \pi_S(\cdot \mid x),$$

the log-ratio of teacher to student on the student's own sample. Its expectation under the student's distribution is $-\mathrm{KL}(\pi_S \,\|\, \pi_T)$ per token, so maximizing it as an RL reward minimizes the reverse KL from student to teacher, token by token, on the student's distribution. Compared with RL, there are many bits per sample (a number at every token) instead of one; compared with SFT, there is no exposure bias, since the contexts are the student's own. The reverse KL is mode-seeking: the student is pushed to put its mass where the teacher's is, and is not penalized for omitting teacher modes it cannot reach, which is what one wants of a smaller student. The requirement is that teacher and student share a tokenizer, so that π_T can be evaluated on the student's tokens, or that the teacher's distribution be mapped onto the student's vocabulary.

The origin is GKD, generalized knowledge distillation (Agarwal et al. 2023), which proposed on-policy sampling with a per-token divergence to the teacher and showed it beat SFT distillation on summarization, translation and reasoning. Qwen3 made it a production stage: its small models are trained by strong-to-weak

distillation, first off-policy (SFT on teacher outputs) and then on-policy, and the report's Table 21 gives the compute for Qwen3-8B, 1,800 GPU hours for the distillation against 17,920 for reinforcement learning on the same model, with better results from distillation. Thinking Machines' October 2025 post (†) reports order-of-magnitude savings against RL for reasoning and for personalization, and is the source of the term's current popularity. Multi-teacher on-policy distillation (2026) is the pattern of slide 27's last row: several specialist teachers trained by RL (one on math, one on code, one on agents), and one student that samples and is scored by whichever teacher is best for the prompt; the MOPD paper (Ma et al. 2026) reports it for MiMo-V2-Flash; DeepSeek-V4 and Nemotron 3 Ultra are reported to use the same pattern (†, commentary), and the compute claims are the labs' own.

Terms

on-policy distillation (OPD) Training a student on its own samples with a per-token reward given by the teacher's log-probabilities, equivalently minimizing the reverse KL to the teacher on the student's distribution.

dense reward A reward at every token, as opposed to one per response. OPD's reward is dense; RLVR's is sparse.

reverse KL $\mathrm{KL}(\pi_S \,\|\, \pi_T)$, the divergence with the student's distribution first; mode-seeking, in that the student is not penalized for missing teacher modes.

GKD Generalized Knowledge Distillation (Agarwal et al. 2023): on-policy distillation with a choice of divergence, the origin of the method.

strong-to-weak distillation Qwen3's term for distilling its large models into small ones, off-policy then on-policy, replacing RL for the small sizes.

multi-teacher OPD On-policy distillation from several specialist teachers into one student, with the teacher chosen per prompt; the 2026 way of merging specialist RL runs.

tokenizer sharing The requirement that teacher and student use the same vocabulary, so that the teacher's probability of the student's tokens can be evaluated.

consolidation Combining the behaviors of several specialist models into one general model, by SFT on their outputs or by OPD.

Intuition

SFT is copying the teacher's homework; RL is doing your own and getting one mark at the end; on-policy distillation is doing your own with the teacher looking over your shoulder and commenting on every line. The last is the most information per hour of work, which is why it is becoming the step that turns several specialists' lessons into one student.

9.3 Slide 29: Evaluating each stage, and the regressions to watch

Every stage is checked by its own suite, and the previous suites are re-run, because the usual damage is elsewhere.

What the slide shows Six chevrons, one per stage, each with its evaluation suite: SFT (IFEval, MT-Bench, GSM8K and MATH, MMLU to see what was forgotten); preference (AlpacaEval 2 length-controlled, Arena-Hard, answer length); RL (AIME avg@32, LiveCodeBench, GPQA Diamond, pass@k); context (NIAH, RULER at every length, the short suite again); safety (XSTest, HarmBench, violation against false-refusal rates); tools (BFCL, τ-bench, SWE-bench Verified). Three cards: regressions, reward hacking, protocol.

Details The slide's rule is in its subtitle: each stage is checked by its own suite, and every earlier suite is re-run. The reason is that a stage's gains are usually visible on its own benchmarks, where they were optimized, and its damage is elsewhere. SFT improves instruction following (IFEval) and conversational quality (MT-Bench, a set of multi-turn questions judged by GPT-4) and can lower MMLU, the knowledge benchmark, which is why MMLU is in the SFT row. Preference optimization raises win rates on AlpacaEval 2 and Arena-Hard, both judged by a model against a reference, and the length-controlled version of AlpacaEval exists because the raw win rate can be raised by writing longer answers; answer length is listed as a metric for the same reason. RL is scored by AIME averaged over 32 samples (slide 18), LiveCodeBench (coding problems collected after the model's training cutoff, to avoid contamination), GPQA Diamond (graduate-level science questions written to be search-proof), and pass@k to see whether diversity survived. Context, safety and tools have the suites of their own sections.

The regressions card is a list of what has gone wrong in practice. Length creep: every judged metric rewards length unless controlled. Over-refusal after safety training. Knowledge loss on MMLU after RL that was run long. Language mixing in reasoning models. Format collapse, where a model stops using the chat template correctly after a stage trained on differently formatted data. And the general case, a drop on the previous stage's suite, which only re-running it detects.

The reward-hacking card is for evaluations as much as for training. A judge model rewards length, confidence and its own style, so a policy trained against one judge can score well on an evaluation by a similar judge without being better; the defence is to read samples, not only scores, and to hold out a judge the policy never saw. Tests can be edited and constraints met with nonsense, as on slide 13, and evaluations that execute code need the same sandboxing as training.

The protocol card is the discipline that makes numbers comparable across stages. Decoding settings (temperature, maximum tokens, system prompt) fixed and identical at every stage, since a change in temperature can move AIME by more than a stage does. Averages over samples and seeds, with variance reported. Held-out prompts, and contamination checks against the training data, since synthetic SFT and RL data are generated at scale and benchmark problems leak into them easily.

Terms

MT-Bench Zheng et al. 2023: 80 multi-turn questions scored by GPT-4 as a judge; the paper also introduced Chatbot Arena and studied judge biases.

AlpacaEval 2 (LC) Dubois et al. 2024: a judged win rate against a reference model on 805 prompts, with a length-controlled version that removes the advantage of longer answers.

Arena-Hard Li et al. 2024: 500 hard prompts drawn from Chatbot Arena, judged pairwise against a baseline by a model; built to agree with human Arena rankings.

LiveCodeBench Jain et al. 2024: a contamination-resistant coding benchmark that continuously collects new problems from contest sites and reports results by date window.

GPQA Graduate-level Google-Proof Q&A (Rein et al. 2023): 448 expert-written science questions on which non-experts with web access score about 34%; Diamond is the 198-question hardest subset.

MMLU Massive Multitask Language Understanding (Hendrycks et al. 2020): 57 subjects of multiple-choice questions; the standard knowledge benchmark and the one that detects forgetting.

length-controlled win rate A judged win rate adjusted by a regression on the length difference between the two responses, so that longer does not win by itself.

contamination The presence of benchmark problems, or close paraphrases, in training data, which inflates scores. A risk for every synthetic data pipeline.

held-out judge A judge model not used during training, so that a policy optimized against one judge's preferences is scored by another's.

decoding settings Temperature, top-p, maximum tokens and system prompt at evaluation time, which must be fixed across stages for scores to be comparable.

Intuition

Each stage is a renovation of one room, and the evaluation suite for that room will say the renovation went well. The question is whether the plumbing in the other rooms still works, and nobody finds out without walking through the whole house again. Re-running every earlier suite after every stage is the walk-through.

9.4 Slide 30: A practical plan for a small team

One 7–8B base, open data, one node of GPUs: the whole pipeline in order, with indicative settings.

What the slide shows Five numbered steps. (1) Base and evaluations first: a 7–8B base with a known chat template; freeze the evaluation suite before training. (2) SFT: 100k–1M examples (the Tülu 3 mix, OpenThoughts3); full fine-tune on 8×80 GB, else LoRA; mask prompts. (3) Preference: DPO on 50–300k pairs (UltraFeedback, the Tülu 3 preference mix); length-normalized if answers grow. (4) RLVR: GRPO on 10–50k checkable prompts; vLLM rollouts; start from the DPO model. (5) Safety, tools, release: 5–10% of the SFT and DPO mix; a guard model at serving; re-run every suite, then loop. Right, a table of indicative hyperparameters for SFT, DPO and GRPO (learning rate, length, batch, key knobs, compute), with a note on their sources, and a list of frameworks.

Details The plan is the deck's recipe reduced to what one team with one node can run, and every number on it is marked indicative: they are the ranges reported by Tülu 3, DAPO and Dr. GRPO and the defaults of the frameworks, and all of them are tuned per model and dataset.

Step 1 is the one teams skip. The base must have a chat template the team understands, because the SFT data will be rendered through it; and the evaluation suite (slide 29) must be fixed before any training, because a suite chosen after seeing results measures the choosing. Step 2, SFT: an open mix such as Tülu 3's or OpenThoughts3 (Guha et al. 2025, a reasoning SFT set built by ablating every pipeline choice), 100k to 1M examples, full fine-tuning if the node holds it (a 7–8B model with Adam states fits on 8×80 GB with sharding), LoRA otherwise; the prompt tokens masked. Learning rate 5×10^{-6} to 2×10^{-5}, one to three epochs, 64–256 sequences per batch; hours per 100k examples on eight GPUs. Step 3, DPO: 50–300k pairs from UltraFeedback or the Tülu 3 preference mix, one epoch, learning rate 5×10^{-7} to 10^{-6}, β 0.05–0.1 (or 5 with Tülu 3's length normalization), 32–128 pairs per batch; hours. Length-normalized DPO if answers grow, which they will. Step 4, GRPO: 10–50k prompts with checkable answers (math with gold answers, code with tests, IFEval-style constraints), starting from the DPO model, rollouts from vLLM, $G = 8$–16, learning rate 10^{-6}, $\varepsilon = 0.2$ (upper 0.28 if following DAPO), β from 0 to 0.01, response lengths of 4–8k tokens, 128–512 prompts per step, 500–2000 steps; days, with the rollouts dominating. Step 5 is the remaining three behaviors handled as data: safety examples and tool-calling transcripts at 5–10% of the SFT and DPO mixes, a guard model at serving rather than more refusal training, and then the full evaluation suite before anything ships. "Then loop" means that the second round starts from the first round's model with rejection-sampled data, as Llama 3 did six times.

The frameworks are the practical choice. TRL (Hugging Face) has SFT, DPO and GRPO trainers and is the simplest to start with; OpenRLHF, veRL and slime are the RL frameworks of slide 14 and run GRPO with vLLM or SGLang rollouts at scale; Axolotl and LLaMA-Factory are configuration-driven fine-tuning tools that cover SFT and DPO for most model families.

Terms

OpenThoughts3 Guha et al. 2025: a 1.2M-example reasoning SFT set and the ablation study that produced it, covering question sourcing, filtering and teacher choice.

UltraFeedback A large open preference dataset (Cui et al. 2023) of model responses to prompts scored by GPT-4 on several criteria, from which pairs are formed.

learning rate The step size of the optimizer. Post-training uses rates one to two orders of magnitude below pretraining, lowest for DPO.

batch size The number of sequences, pairs or prompts per optimizer step; for GRPO, prompts times G is the number of rollouts per step.

TRL Transformer Reinforcement Learning, Hugging Face's library with SFT, reward-model, DPO and GRPO trainers.

Axolotl / LLaMA-Factory Configuration-driven fine-tuning tools that support SFT, DPO and LoRA across model families.

freeze the suite Fix the set of evaluations and their protocol before training begins, so that results cannot be selected after the fact.

Intuition

The plan is the whole deck in five lines, and the only parts of it that need a GPU cluster are the rollouts in step 4. Everything else runs on one node in hours. What separates a good run from a bad one is step 1 (knowing what you will measure) and the last words of step 5 (measuring it again).

10 Wrap-up (slides 31–32)

10.1 Slide 31: Summary: what each stage adds, and what it costs

Post-training is a sequence of cheap stages on top of one expensive one; each adds a behavior and a risk.

What the slide shows Six chevrons, one per stage, each with what it adds and what it risks. SFT adds format, instruction following and tool syntax; risks forgetting and copying the data's style. Preference adds taste, helpfulness and refusals; risks length creep and over-optimization. RL (RLVR) adds reasoning and accuracy where checkable; risks entropy collapse and reward hacking. Context adds 128k windows; risks short-context regression. Safety adds refusal where it should; risks over-refusal and an alignment a few tokens deep. Tools add calls and agents; risk invented calls and credit assignment. Below, five things to remember.

Details The five points are the deck's argument, and each one points back to the slides that made it.

One: SFT is pretraining loss on the answer tokens (slide 4); everything after it is a way to learn from a comparison or a reward instead of a demonstration (slide 3). This is the organizing idea, and it says why the later stages exist: a demonstration can only teach what someone could write.

Two: the reward model is a proxy and is optimized past its validity without a leash (slides 8 and 9); verifiable rewards remove the proxy where a program can check the answer (slide 13), which is why RL moved to math, code and tools first (slides 16 and 26). Everything about the 2025–26 shift follows from the existence of an un-hackable reward on some tasks.

Three: GRPO is PPO without the critic, built for one reward per answer (slide 12); the engineering problem is the rollout engine, not the gradient (slide 14). The algorithm is a page; the system is the work.

Four: reasoning RL mostly sharpens what the base model can already sample (slide 18); distillation moves it between models cheaply (slide 17), and on-policy distillation is becoming the default middle stage (slide 28). A team deciding where to spend compute should know that the search happens once, in the largest model, and is copied everywhere else.

Five: context, safety and tool use are each a data problem with their own evaluations (Sections 5–8); each stage is checked by re-running every earlier suite (slide 29). The behaviors are different; the protocol for adding one is the same.

Terms

stage One training phase of a recipe, with its own data, objective and evaluation; the deck's unit of organization.

regression A loss of performance on an earlier stage's evaluations caused by a later stage. The thing every re-run suite is looking for.

Intuition

A base model is expensive and knows a great deal. Every stage on top of it is cheap and teaches one habit, and every habit has a characteristic way of going wrong. Post-training is the practice of adding habits one at a time and checking, after each, that the earlier ones are still there.

10.2 Slide 32: Sources

Primary papers, checked against arXiv; † marks claims taken from release notes, blogs or commentary.

What the slide shows Two columns of citations grouped by section: SFT and instruction data; preference; RL and reasoning; context; safety; tools and agents; distillation and 2026 recipes; evaluation. Each entry gives authors, year, short title and arXiv identifier. A closing note says that curves labelled illustrative are

sketches of a mechanism rather than data, that numbers labelled derived were computed from quoted ones, and that the hyperparameter ranges on the plan slide are indicative.

Details The full reading list, one entry per paper with a sentence on why it is there, is Appendix D of these notes, and matches the paper archive that accompanies them. Three conventions carry over from the slide. Numbers quoted from a paper were checked against that paper's text or tables, and the table is named where it matters (GCG's Table 2, Qwen3's Table 21, InstructGPT's Table 6). Claims marked † come from sources that are not papers: OpenAI's o1 and SWE-bench Verified posts, bloc97's NTK-aware post, Anthropic's many-shot report, the Llama Guard 3 model card, Thinking Machines' on-policy distillation post, and the release notes and commentary on the 2026 models (OLMo 3, DeepSeek-V4, GLM-5, MiMo-V2-Flash, Kimi K2.5, Nemotron 3); they should be read as what the labs said, not as verified results. And every plot labelled illustrative was drawn to show the shape of a mechanism, following the form of a cited figure where one is named (Gao et al. 2022, R1-Zero's Figure 3, Yue et al. 2025, Liu et al. 2023, Qi et al. 2023 and 2024); none of them is data, and the real figures are in the papers.

Terms

illustrative The deck's label for a plot drawn to show the shape of a mechanism, not reproduced from data.

derived The deck's label for a number computed from quoted numbers (for instance a standard deviation from a set of rewards).

† The deck's mark for a claim taken from a release note, blog post, model card or commentary rather than a paper.

Intuition

The sources slide is a map of how much to trust each claim in the deck: a table number means it was checked, a dagger means it was reported, and "illustrative" means it was drawn. Students should carry the same three-way distinction into their own reading of model releases.

A Notation

symbol	meaning	notes
x, y, y_t	prompt, response, its t-th token	$y_{<t}$ is the prefix; $\lvert y\rvert$ the length; T the last position
π_θ	the policy being trained	the model as a distribution over responses
π_{ref}	the reference policy	frozen; usually the SFT model; the anchor of every KL term
$\pi_{\theta_{\text{old}}}$	the sampling policy	the policy that generated the current batch of rollouts
π_T, π_S	teacher, student	in distillation
y_w, y_l	chosen, rejected	the winner and loser of a comparison
$r(x,y)$, r_ϕ	reward, learned reward model	scalar per response
r_t	per-token reward	PPO: RM at $t=T$ minus $\beta\,\text{KL}_t$; OPD: teacher minus student log-prob
$\hat{A}$, $\hat{A}_t$, $\hat{A}_i$	advantage	per token (PPO), per response i in a group (GRPO)
V_ψ	value model	the critic; absent in GRPO, RLOO, DPO
ρ_t, $\rho_{i,t}$	importance ratio	$\pi_\theta / \pi_{\theta_{\text{old}}}$ at token t
ε	clip range	0.2; DAPO upper 0.28
β	KL coefficient	PPO 0.01–0.1; DPO 0.1 (5 length-normalized); GRPO 0 to 0.01
G, K	group size, samples per prompt	GRPO G 8–64; rejection sampling K
$\text{KL}(\pi \,\|\, \pi')$	Kullback–Leibler divergence	$\mathbb{E}_{y\sim\pi}[\log \pi(y) - \log \pi'(y)]$
d	$\sqrt{\text{KL}(\pi \,\|\, \pi_{\text{init}})}$	Gao et al.'s distance; also head dimension in RoPE, by context
$Z(x)$	partition function	normalizer of the optimal policy; cancels in DPO
σ	logistic function	$1/(1+e^{-z})$
θ_i, b, s, L	RoPE frequency, base, extension factor, trained length	$\theta_i = b^{-2i/d}$; extend $L \to sL$
t (YaRN)	attention temperature	$\sqrt{1/t} = 0.1 \ln s + 1$
pass@k, avg@n, pass^k	evaluation metrics	any of k; mean of n; all of k

B Formulas at a glance

SFT. $\mathscr{L}_{\text{SFT}} = -\sum_{t \in \text{assistant}} \log \pi_\theta(y_t \mid x, y_{<t})$; system, user and tool tokens masked.

Bradley–Terry and the reward model. $P(y_w \succ y_l) = \sigma(r_w - r_l)$; $\mathscr{L}_{\text{RM}} = -\log \sigma\big(r_\phi(x, y_w) - r_\phi(x, y_l)\big)$. Rewards are defined up to a constant per prompt.

Overoptimization (Gao et al.). $R_{\text{RL}}(d) = d(\alpha - \beta \log d)$ for the gold reward, $d = \sqrt{\text{KL}(\pi \,\|\, \pi_{\text{init}})}$; the proxy rises monotonically; the peak moves out with RM size and data.

KL-regularized objective. Maximize $\mathbb{E}_{y\sim\pi}[r(x,y)] - \beta\,\text{KL}(\pi \,\|\, \pi_{\text{ref}})$; optimum $\pi^*(y \mid x) = \pi_{\text{ref}}(y \mid x)\, e^{r(x,y)/\beta}/Z(x)$; inverse: $r = \beta \log(\pi^*/\pi_{\text{ref}}) + \beta \log Z$.

PPO. $\mathscr{L} = \mathbb{E}_t[\min(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t)]$, $\rho_t = \pi_\theta(y_t \mid \cdot)/\pi_{\theta_{\text{old}}}(y_t \mid \cdot)$; $r_t = r_\phi(x,y)\mathbb{1}[t = T] - \beta\,\text{KL}_t$ with $\text{KL}_t = \log \pi_\theta(y_t \mid \cdot) - \log \pi_{\text{ref}}(y_t \mid \cdot)$; advantages by GAE from V_ψ.

REINFORCE and RLOO. $\nabla \log \pi(y \mid x)\,(r - b)$; RLOO baseline $b_i = \frac{1}{K-1}\sum_{j\neq i} r_j$.

DPO. $\mathscr{L}_{\text{DPO}} = -\mathbb{E}\big[\log \sigma\big(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\big)\big]$; implicit reward $\beta \log(\pi_\theta/\pi_{\text{ref}})$. Length-normalized: divide each log-ratio by $|y|$.

GRPO. $\hat{A}_i = (r_i - \text{mean}\, r)/\text{std}\, r$ over the group; $\mathscr{J} = \mathbb{E}\big[\frac{1}{G}\sum_i \frac{1}{|y_i|}\sum_t \min(\rho_{i,t}\hat{A}_i, \text{clip}(\rho_{i,t}, 1 \pm \varepsilon)\hat{A}_i) - \beta\, \mathbb{D}_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]\big]$; KL estimator $\pi_{\text{ref}}/\pi_\theta - \log(\pi_{\text{ref}}/\pi_\theta) - 1$. Dr. GRPO drops $1/|y_i|$ and the std; DAPO uses $\varepsilon_{\text{high}} = 0.28$, token-level averaging, dynamic sampling, $\beta = 0$.

Verifiable rewards. math: $\mathbb{1}[\text{norm}(\hat{a}) = \text{norm}(a^*)]$ (+ format); code: fraction of tests passed, or $\mathbb{1}[\text{all pass}]$; instructions: fraction of constraints satisfied.

Pass@k. unbiased from n samples with c correct: $1 - \binom{n-c}{k}/\binom{n}{k}$. Pass^k (all of k): p^k for independent trials with success p.

RoPE. $\theta_i = b^{-2i/d}$, $i = 0, \ldots, d/2-1$; angle $m\theta_i$ at position m; wavelength $2\pi/\theta_i$. PI: $\theta_i' = \theta_i/s$. NTK-aware: $b' = b\, s^{d/(d-2)}$. YaRN: $\theta_i' = \theta_i\big((1-\gamma_i)/s + \gamma_i\big)$ with ramp γ_i from 1 (short wavelength) to 0 (long); logits scaled by $1/\sqrt{t}$, $\sqrt{1/t} = 0.1 \ln s + 1$.

On-policy distillation. $r_t = \log \pi_T(y_t \mid x, y_{<t}) - \log \pi_S(y_t \mid x, y_{<t})$ on $y \sim \pi_S$; $\mathbb{E}[\sum_t r_t] = -\text{KL}(\pi_S \,\|\, \pi_T)$ (reverse KL, mode-seeking).

Worked numbers (derived). GRPO group $(1,0,1,0,0)$: mean 0.4, std $\sqrt{0.24} = 0.49$, $\hat{A} = +1.22/-0.82$. Attention cost ratio 8k $\to$ 128k: $16^2 = 256$. RoPE wavelength of the lowest pair at $b = 10^4$: $2\pi \cdot 10^4 \approx 63{,}000$ tokens; at $b = 5 \times 10^5$: ≈ 3.1M.

C Exercises

1. **Masking.** A multi-turn SFT example has a 40-token system prompt, three user turns of 30 tokens each, two tool outputs of 400 tokens each, and three assistant turns of 60 tokens each. What fraction of the sequence carries loss? If the tool outputs were accidentally unmasked, what would the model learn to do, and how would you detect it at evaluation?

2. **LoRA accounting.** A 7B model has 32 layers, each with four attention projections and three MLP projections of hidden size 4096 (MLP intermediate 11008). Count the parameters of rank-16 LoRA adapters on the attention projections only, and on all seven; express both as a fraction of 7B. Why does the optimizer-state saving matter more than the parameter count?

3. **Bradley–Terry.** Show that adding a prompt-dependent constant $c(x)$ to $r_\phi(x,\cdot)$ leaves $\mathscr{L}_{\text{RM}}$ unchanged. If a reward model gives $r_w = 2.1$ and $r_l = -0.4$, what probability does it assign to the chosen response winning? What reward gap corresponds to 90%?

4. **Overoptimization.** With $R_{\text{RL}}(d) = d(\alpha - \beta \log d)$, find the d at which the gold reward peaks and its value there. For $\alpha = 1$, $\beta = 0.25$, compute both, and the KL (in nats) at the peak. What does a target-KL controller in PPO have to know to stop there?

5. **DPO derivation.** Starting from $\max_\pi \mathbb{E}_{y\sim\pi}[r(x,y)] - \beta \text{KL}(\pi\|\pi_{\text{ref}})$ for a fixed x, write the Lagrangian with the constraint $\sum_y \pi(y) = 1$, take the derivative with respect to $\pi(y)$, and show the optimum is $\pi_{\text{ref}}(y)e^{r(y)/\beta}/Z$. Then show that substituting the inverted reward into the Bradley–Terry loss gives $\mathscr{L}_{\text{DPO}}$ with Z cancelled.

6. **DPO gradient.** Differentiate $\mathscr{L}_{\text{DPO}}$ with respect to θ and show it has the form $-\beta\,\sigma(\hat{r}_l - \hat{r}_w)\,[\nabla \log \pi_\theta(y_w) - \nabla \log \pi_\theta(y_l)]$ with $\hat{r} = \beta \log(\pi_\theta/\pi_{\text{ref}})$. Interpret the weight: when is a pair nearly ignored?

7. **GRPO advantages.** A group of 8 rewards is $(1,1,1,0,0,0,0,0)$. Compute the advantages with and without the standard-deviation normalization. Repeat for $(1,0,0,0,0,0,0,0)$ and $(1,1,1,1,1,1,1,0)$. Which prompts contribute most gradient under each scheme, and what is Dr. GRPO's objection?

8. **Length bias.** Two wrong responses in a group have advantage -0.5 and lengths 200 and 2000 tokens. Under GRPO's $1/|y_i|$ normalization, what is the total gradient weight on each? Under a token-level loss with the batch's total token count as the denominator? Explain in one sentence why the first scheme lets long wrong answers grow.

9. **Rollout arithmetic.** An RL step uses 512 prompts, $G = 16$, and responses averaging 4,000 tokens. How many tokens are generated per step? If the rollout engine sustains 20,000 tokens per second across the node and the gradient step takes 90 seconds, what fraction of the step is generation? What does asynchronous generation one step ahead change?

10. **Zero-advantage prompts.** If the policy's pass rate on a prompt is p and $G = 8$, what is the probability that all 8 samples agree (all right or all wrong)? Plot it against p and say which prompts dynamic sampling discards most. What does this imply about choosing prompts whose base-model pass rate is near 0 or 1?

11. **Pass@k.** A problem has been sampled $n = 64$ times with $c = 8$ correct. Compute pass@1, pass@8 and pass@64 with the unbiased estimator. A second model has $c = 40$ of 64. Compare the two at $k = 1$ and $k = 64$, and relate the comparison to Yue et al.'s sharpening claim.

12. **Budget forcing.** A model's accuracy rises as $a(n) = 0.15 + 0.70\,/\,(1 + e^{-1.6(\ln n - \ln 4000)})$ with n thinking tokens (the deck's illustrative curve). Compute a at 500, 2,000, 8,000 and 32,000 tokens. At what budget is 90% of the gain over direct answering reached, and what does doubling the budget beyond that buy?

13. **RoPE wavelengths.** For $d = 128$ and $b = 10^4$, compute the wavelength $2\pi/\theta_i$ for $i = 0, 16, 32, 48, 63$. Which pairs have wavelength longer than a 4k training window? Repeat for $b = 5 \times 10^5$. Under YaRN with $s = 16$, which pairs are interpolated, which are left alone, and which are ramped, if the ramp runs between wavelengths of $L/32$ and L?

14. **Context cost.** Pretraining at 8k costs C in attention FLOPs per token. What does 128k cost per token? If the extension stage uses 800B tokens at lengths averaging 64k and pretraining used 15T tokens at 8k, what fraction of the total attention FLOPs is the extension stage? (Assume attention cost per token is proportional to length.)

15. **Refusal trade-off.** The deck's illustrative curve is $v(f) = 1 + 32e^{-f/7}$, violations in percent against false refusals in percent. Find the false-refusal rate at which violations fall to 5%, and the slope there. If a product requires violations below 3%, what false-refusal rate results, and what would you need to change to do better than the curve?

16. **Shallow alignment.** The KL between aligned and base models at token t is $1.6e^{-(t-1)/2.2} + 0.03$ (illustrative). What fraction of the total KL over the first 30 tokens is in the first 3? Explain why a prefix attack that fixes 3 tokens leaves so little, and what recovery-data training is meant to change in this curve.

17. **Function-calling data.** Design a verification pipeline for synthetic function-calling examples with three stages (parse, execute, judge). For each stage, give one example that passes the stage but should be rejected by the next. Why are negative examples (no tool needed) necessary, and how would you generate them?

18. **Agentic credit.** A trajectory has 40 model turns and succeeds. GRPO gives every turn the same advantage. Propose a turn-level scheme using a value estimate at each turn, state what it needs that GRPO does not, and give one reason it might be worse in practice.

19. **Distillation economics.** Qwen3 reports 1,800 GPU hours for distillation and 17,920 for RL on the same model (Table 21). If a lab trains one 200B teacher by RL at X GPU hours and then distils it into five small models, write the total cost under the two strategies (RL each small model, or distil each) as a function of X and the per-model costs, and find the number of students at which distillation wins even if its per-student cost were equal to RL's.

20. **Reverse KL.** Show that $\mathbb{E}_{y\sim\pi_S}[\sum_t r_t]$ with $r_t = \log \pi_T(y_t \mid \cdot) - \log \pi_S(y_t \mid \cdot)$ equals $-\mathrm{KL}(\pi_S \| \pi_T)$ (sequence-level). Give a two-mode toy teacher and show that a one-mode student minimizing reverse KL picks one mode, while one minimizing forward KL spreads between them.

D Reading list

The papers behind each part of the deck, in the order they are discussed, with one line on why each is worth reading. Entries with an arXiv identifier are included in the paper archive; entries without one are listed with a URL. Items marked *secondary source* are blog posts, model cards or commentary.

1. Overview, SFT and instruction data (slides 1-6)

- **Wei et al. (2021).** *Finetuned language models are zero-shot learners*. arXiv:2109.01652. FLAN: instruction tuning on templated NLP tasks; the first evidence that instructions generalize to unseen tasks.
- **Sanh et al. (2021).** *Multitask prompted training enables zero-shot task generalization*. arXiv:2110.08207. T0: the same idea from the open community (PromptSource), with the prompt-template collection that later mixes reuse.
- **Ouyang et al. (2022).** *Training language models to follow instructions with human feedback*. arXiv:2203.02155. InstructGPT: the SFT -> reward model -> PPO pipeline with 13k/33k/31k prompts; a 1.3B RLHF model preferred over 175B GPT-3. The template for everything after.
- **Wang et al. (2022).** *Self-Instruct: aligning language models with self-generated instructions*. arXiv:2212.10560. Bootstrap 52k instruction examples from a model's own generations; the method behind Alpaca and most synthetic SFT data.
- **Taori et al. (2023).** *Stanford Alpaca: an instruction-following LLaMA model (GitHub)*. `https://github.com/tatsu-lab/stanford_alpaca`. 52K Self-Instruct examples from text-davinci-003 fine-tuning LLaMA 7B; the open instruction-tuning moment of 2023. Not on arXiv.
- **Zhou et al. (2023).** *LIMA: less is more for alignment*. arXiv:2305.11206. 1,000 curated examples suffice for format and style: the 'superficial alignment' argument.
- **Lambert et al. (2024).** *Tulu 3: pushing frontiers in open language model post-training*. arXiv:2411.15124. The fully open recipe: 939k-prompt SFT mix, length-normalized DPO, and RLVR, with every dataset and hyperparameter published.
- **Hu et al. (2021).** *LoRA: low-rank adaptation of large language models*. arXiv:2106.09685. Low-rank adapters on frozen weights; how most SFT is done without the memory of full fine-tuning.
- **Dettmers et al. (2023).** *QLoRA: efficient finetuning of quantized LLMs*. arXiv:2305.14314. 4-bit base plus LoRA adapters: a 65B model fine-tuned on one 48 GB GPU.
- **Zelikman et al. (2022).** *STaR: bootstrapping reasoning with reasoning*. arXiv:2203.14465. Keep the rationales that reach the right answer and fine-tune on them, repeat: rejection sampling as self-improvement, the bridge from SFT to RL.
- **Bai et al. (2022).** *Training a helpful and harmless assistant with reinforcement learning from human feedback*. arXiv:2204.05862. Anthropic's HH-RLHF: the data-collection protocol, the helpfulness/harmlessness tension, and the open preference dataset.

2. Preference optimization: RLHF, reward models, PPO, DPO (slides 7-11)

- **Christiano et al. (2017).** *Deep reinforcement learning from human preferences*. arXiv:1706.03741. Learning a reward model from pairwise human comparisons and optimizing it with RL; the origin of RLHF.

- **Ziegler et al. (2019).** *Fine-tuning language models from human preferences*. arXiv:1909.08593. RLHF applied to GPT-2 for stylistic continuation and summarization; the KL-to-reference penalty appears here.
- **Stiennon et al. (2020).** *Learning to summarize from human feedback*. arXiv:2009.01325. RLHF for summarization with the reward model, PPO and KL leash in the form InstructGPT reused; also an early overoptimization plot.
- **Bradley and Terry (1952).** *Rank analysis of incomplete block designs: I. The method of paired comparisons*. `https://doi.org/10.2307/2334029`. The pairwise-comparison model behind every reward model: P(a > b) = sigma(r_a - r_b). Biometrika 39(3/4). Not on arXiv.
- **Gao, Schulman and Hilton (2022).** *Scaling laws for reward model overoptimization*. arXiv:2210.10760. Gold-vs-proxy reward curves and the fits R(d) = d(alpha - beta log d) in d = sqrt(KL); why the reward model cannot be optimized forever.
- **Schulman et al. (2017).** *Proximal policy optimization algorithms*. arXiv:1707.06347. The clipped-ratio policy gradient (epsilon = 0.2) that RLHF and GRPO both inherit.
- **Rafailov et al. (2023).** *Direct preference optimization: your language model is secretly a reward model*. arXiv:2305.18290. The closed-form solution of the KL-regularized objective turned into a classification loss on pairs; no reward model, no sampling.
- **Azar et al. (2023).** *A general theoretical paradigm to understand learning from human preferences*. arXiv:2310.12036. IPO: a bounded alternative to DPO's loss that resists overfitting to deterministic preferences.
- **Ethayarajh et al. (2024).** *KTO: model alignment as prospect theoretic optimization*. arXiv:2402.01306. Alignment from unpaired thumbs-up/thumbs-down labels, using a prospect-theory-shaped loss.
- **Hong, Lee and Thorne (2024).** *ORPO: monolithic preference optimization without reference model*. arXiv:2403.07691. SFT and preference in one loss, no reference model.
- **Meng, Xia and Chen (2024).** *SimPO: simple preference optimization with a reference-free reward*. arXiv:2405.14734. Length-normalized log-probability as the implicit reward, no reference model; strong on AlpacaEval 2 LC.
- **Ahmadian et al. (2024).** *Back to basics: revisiting REINFORCE-style optimization for learning from human feedback in LLMs*. arXiv:2402.14740. RLOO: REINFORCE with a leave-one-out baseline matches PPO without a value model; the case for simpler on-policy RL.
- **Cui et al. (2023).** *UltraFeedback: boosting language models with scaled AI feedback*. arXiv:2310.01377. The 64k-prompt AI-labelled preference dataset behind most open DPO recipes (Zephyr, Tulu).

3. RL with verifiable rewards and RL systems (slides 12-14)

- **Shao et al. (2024).** *DeepSeekMath: pushing the limits of mathematical reasoning in open language models*. arXiv:2402.03300. Introduces GRPO: group-normalized advantages, no value model, KL added to the loss with an unbiased estimator.
- **Yu et al. (2025).** *DAPO: an open-source LLM reinforcement learning system at scale*. arXiv:2503.14476. Clip-higher, dynamic sampling, token-level loss and overlong reward shaping; 50 on AIME 2024 from Qwen2.5-32B, fully open.
- **Liu et al. (2025).** *Understanding R1-Zero-like training: a critical perspective*. arXiv:2503.20783. Dr. GRPO: the length and difficulty biases of GRPO's normalizations, and the unbiased fix.

- **Sheng et al. (2024).** *HybridFlow: a flexible and efficient RLHF framework*. arXiv:2409.19256. veRL: the trainer/rollout-engine architecture, single-controller programming model, and the systems view of slide 14.
- **Hu et al. (2025).** *Open-Reasoner-Zero: an open source approach to scaling up reinforcement learning on the base model*. arXiv:2503.24290. Vanilla PPO with rule-based rewards and no KL reproduces R1-Zero-style scaling on a base model; everything released.
- **Ling Team (Ant Group) (2025).** *Ring-1T report and the IcePop train/inference mismatch fix (release)*. `https://huggingface.co/inclusionAI/Ring-1T`. IcePop: mask the tokens whose trainer and rollout-engine probabilities diverge most. Secondary source; cited in the deck from the release.

4. Reasoning: test-time compute, R1, distillation, budgets (slides 15-18)

- **Wei et al. (2022).** *Chain-of-thought prompting elicits reasoning in large language models*. arXiv:2201.11903. Ask for steps, get better answers; the capability was latent in large base models.
- **Lightman et al. (2023).** *Let's verify step by step*. arXiv:2305.20050. Process supervision and PRM800K: step-level reward beats outcome-only on MATH (78.2 vs 72.4).
- **Snell et al. (2024).** *Scaling LLM test-time compute optimally can be more effective than scaling model parameters*. arXiv:2408.03314. Compute-optimal test-time strategies; inference compute can beat a 14x larger model on some problems.
- **DeepSeek-AI (2025).** *DeepSeek-R1: incentivizing reasoning capability in LLMs via reinforcement learning*. arXiv:2501.12948. R1-Zero (base model + GRPO + rule rewards), the four-stage R1 pipeline, and the distilled Qwen and Llama students.
- **Kimi Team (2025).** *Kimi k1.5: scaling reinforcement learning with LLMs*. arXiv:2501.12599. The other January-2025 reasoning-RL report: long-context RL, length penalties and long-to-short distillation.
- **Muennighoff et al. (2025).** *s1: simple test-time scaling*. arXiv:2501.19393. 1,000 curated traces and 'Wait' budget forcing turn Qwen2.5-32B into a reasoner; thinking length as a dial.
- **Qwen Team (2025).** *Qwen3 technical report*. arXiv:2505.09388. Four post-training stages, thinking-mode fusion (/think and /no_think), thinking budgets, strong-to-weak distillation.
- **Yue et al. (2025).** *Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?*. arXiv:2504.13837. Pass@k at large k barely moves: current RL sharpens what the base could already sample.
- **Guha et al. (2025).** *OpenThoughts: data recipes for reasoning models*. arXiv:2506.04178. The open 1.2M-example reasoning SFT dataset and what made it work.
- **OpenAI (2024).** *Learning to reason with LLMs (blog post, September 2024)*. `https://openai.com/index/learning-to-reason-with-llms/`. The o1 announcement: RL on verifiable rewards teaches the model to think long. Secondary source.

5. Long context (slides 19-21)

- **Su et al. (2021).** *RoFormer: enhanced transformer with rotary position embedding*. arXiv:2104.09864. RoPE: the rotation-by-position encoding that every extension method rescales.

- **Chen et al. (2023).** *Extending context window of large language models via positional interpolation*. arXiv:2306.15595. Positions scaled by L/L': LLaMA 2k -> 32k within 1,000 fine-tuning steps.
- **bloc97 (2023).** *NTK-aware scaled RoPE (Reddit post, r/LocalLLaMA)*. `https://www.reddit.com/r/LocalLLaMA/comments/14lz7j5/`. Change the RoPE base instead of the positions; no fine-tuning needed for modest extension. Secondary source.
- **Peng et al. (2023).** *YaRN: efficient context window extension of large language models*. arXiv:2309.00071. NTK-by-parts interpolation plus attention temperature; 128k with less than 0.1% of the pretraining data.
- **Xiong et al. (2023).** *Effective long-context scaling of foundation models*. arXiv:2309.16039. Adjusted base frequency (10,000 -> 500,000) with continued pretraining; the Llama 2 long-context study.
- **Ding et al. (2024).** *LongRoPE: extending LLM context window beyond 2 million tokens*. arXiv:2402.13753. Per-dimension rescale factors found by evolutionary search; 2048k windows.
- **Grattafiori et al. (2024).** *The Llama 3 herd of models*. arXiv:2407.21783. Six-stage extension 8K -> 128K over ˜800B tokens, the NIAH gate, and 0.1% synthetic long-context SFT data; also the post-training recipe of slide 27.
- **Liu et al. (2023).** *Lost in the middle: how language models use long contexts*. arXiv:2307.03172. The U-shaped curve: facts in the middle of the window are missed.
- **Hsieh et al. (2024).** *RULER: what's the real context size of your long-context language models?*. arXiv:2404.06654. Thirteen tasks in four categories; most models fall short of their claimed length.
- **Kamradt (2023).** *Needle in a haystack: pressure testing LLMs (GitHub)*. `https://github.com/gkamradt/LLMTest_NeedleInAHaystack`. The needle test: hide a sentence, ask for it. Necessary, not sufficient. Not on arXiv.
- **DeepSeek-AI (2024).** *DeepSeek-V3 technical report*. arXiv:2412.19437. Two-phase YaRN extension 4K -> 32K -> 128K, and the post-training pipeline of a 671B MoE.

6. Safety alignment (slides 22-24)

- **Askell et al. (2021).** *A general language assistant as a laboratory for alignment*. arXiv:2112.00861. Helpful, honest, harmless: the targets, and the first measurements of alignment interventions at scale.
- **Bai et al. (2022).** *Constitutional AI: harmlessness from AI feedback*. arXiv:2212.08073. Critique-and-revision SFT, then RL with AI preference labels for harmlessness (RLAIF).
- **Lee et al. (2023).** *RLAIF vs. RLHF: scaling reinforcement learning from human feedback with AI feedback*. arXiv:2309.00267. AI labels match human labels for preference training at a fraction of the cost.
- **Touvron et al. (2023).** *Llama 2: open foundation and fine-tuned chat models*. arXiv:2307.09288. Separate helpfulness and safety reward models, safety context distillation, and the false-refusal trade-off.
- **Rottger et al. (2023).** *XSTest: a test suite for identifying exaggerated safety behaviours in large language models*. arXiv:2308.01263. 250 safe prompts that look unsafe, 200 unsafe contrasts: the over-refusal benchmark.
- **Zou et al. (2023).** *Universal and transferable adversarial attacks on aligned language models*. arXiv:2307.15043. GCG suffixes optimized on open models transfer to closed ones (Table 2: GPT-3.5 86.6%, GPT-4 46.9%, Claude-2 2.1%).

- **Anil et al. (2024).** *Many-shot jailbreaking*. https://proceedings.neurips.cc/paper_files/paper/2024/hash/ea456e232efb72d261715e33ce25f208-Abstract.html. Hundreds of faux dialogues in a long context defeat safety training. NeurIPS 2024; also an Anthropic research page. Not on arXiv.
- **Mazeika et al. (2024).** *HarmBench: a standardized evaluation framework for automated red teaming and robust refusal*. arXiv:2402.04249. 510 behaviors, 18 attacks, 33 models, one scoring protocol.
- **Inan et al. (2023).** *Llama Guard: LLM-based input-output safeguard for human-AI conversations*. arXiv:2312.06674. An LLM classifier for inputs and outputs with a hazard taxonomy; version 3 covers 14 categories.
- **Meta AI (2024).** *Llama Guard 3 model card*. https://huggingface.co/meta-llama/Llama-Guard-3-8B. The 14 hazard categories (S1-S14) aligned with the MLCommons taxonomy. Secondary source.
- **Guan et al. (2024).** *Deliberative alignment: reasoning enables safer language models*. arXiv:2412.16339. Teach the model the text of the safety specification and to reason over it; SFT on spec-citing chains, then RL with a spec-aware judge.
- **Qi et al. (2023).** *Fine-tuning aligned language models compromises safety, even when users do not intend to!*. arXiv:2310.03693. Ten adversarial examples for under $0.20 remove GPT-3.5 Turbo's refusals; benign fine-tuning erodes safety too.
- **Qi et al. (2024).** *Safety alignment should be made more than just a few tokens deep*. arXiv:2406.05946. Alignment lives in the first few output tokens; deepen it with prefix-recovery data and a constrained fine-tuning objective.
- **Li et al. (2024).** *The WMDP benchmark: measuring and reducing malicious use with unlearning*. arXiv:2403.03218. 3,668 hazardous-knowledge questions and RMU unlearning.
- **Tamirisa et al. (2024).** *Tamper-resistant safeguards for open-weight LLMs*. arXiv:2408.00761. Safeguards that survive hundreds of fine-tuning steps; the open-weights problem stated and partly addressed.

7. Tools and agents (slides 25-26)

- **Schick et al. (2023).** *Toolformer: language models can teach themselves to use tools*. arXiv:2302.04761. Self-supervised insertion of API calls into text; the first LLM tool-use training recipe.
- **Yao et al. (2022).** *ReAct: synergizing reasoning and acting in language models*. arXiv:2210.03629. Interleaved thought and action traces; the format every agent loop descends from.
- **Qin et al. (2023).** *ToolLLM: facilitating large language models to master 16000+ real-world APIs*. arXiv:2307.16789. ToolBench: 16,464 RapidAPI endpoints and synthetic multi-tool dialogues for SFT.
- **Liu et al. (2024).** *APIGen: automated pipeline for generating verifiable and diverse function-calling datasets*. arXiv:2406.18518. 60k function-calling examples over 3,673 APIs with three-stage verification; the xLAM models.
- **Patil et al. (2025).** *The Berkeley Function Calling Leaderboard (BFCL): from tool use to agentic evaluation of large language models*. https://proceedings.mlr.press/v267/patil25a/patil25a.pdf. Single, parallel and multi-turn function-calling evaluation. ICML 2025, PMLR 267:48371-48392; no arXiv version.

- **Yao et al. (2024).** *tau-bench: a benchmark for tool-agent-user interaction in real-world domains*. arXiv:2406.12045. Simulated users in retail and airline domains; the passˆk consistency metric.
- **Jimenez et al. (2023).** *SWE-bench: can language models resolve real-world GitHub issues?*. arXiv:2310.06770. Resolve real GitHub issues judged by hidden tests; the environment of coding-agent RL.
- **OpenAI (2024).** *Introducing SWE-bench Verified (blog post, August 2024)*. `https://openai.com/index/introducing-swe-bench-verified/`. The 500 human-validated tasks. Secondary source.
- **Yang et al. (2024).** *SWE-agent: agent-computer interfaces enable automated software engineering*. arXiv:2405.15793. How the interface (tools, observations) shapes agent performance on SWE-bench.
- **Kimi Team (2025).** *Kimi K2: open agentic intelligence*. arXiv:2507.20534. Agentic data synthesis at scale, joint RL with verifiable rewards and a self-critique rubric, MuonClip.
- **NVIDIA (2025).** *NeMo Gym: a library for RL environments (GitHub)*. `https://github.com/NVIDIA-NeMo/Gym`. Environments, verifiers and reward signals for agentic post-training. Secondary source.

8. Open recipes and on-policy distillation (slides 27-28, 30)

- **Agarwal et al. (2023).** *On-policy distillation of language models: learning from self-generated mistakes*. arXiv:2306.13649. GKD: the student samples, the teacher scores every token; reverse KL; the origin of on-policy distillation.
- **Thinking Machines Lab (2025).** *On-policy distillation (blog post, October 2025)*. `https://thinkingmachines.ai/blog/on-policy-distillation/`. Reported order-of-magnitude compute savings against RL for reasoning. Secondary source.
- **Ma et al. (2026).** *MOPD: multi-teacher on-policy distillation for capability integration in LLM post-training*. arXiv:2606.30406. Several specialist teachers into one student, the best teacher chosen per prompt; reported for MiMo-V2-Flash (DeepSeek-V4 and Nemotron 3 Ultra are said in commentary to use the same pattern).
- **Allen Institute for AI (2025).** *OLMo 3 (release notes, November 2025)*. `https://allenai.org/blog/olmo3`. Think, Instruct and RL-Zero variants on one base, SFT -> DPO -> RLVR. Secondary source.
- **Tunstall et al. (2023).** *Zephyr: direct distillation of LM alignment*. arXiv:2310.16944. Distilled SFT plus DPO on UltraFeedback: the small-model recipe of late 2023 that made DPO the open default.
- **Ivison et al. (2023).** *Camels in a changing climate: enhancing LM adaptation with Tulu 2*. arXiv:2311.10702. Tulu 2: DPO at 70B scale with open data; the predecessor of Tulu 3's recipe.

9. Evaluation (slide 29)

- **Zhou et al. (2023).** *Instruction-following evaluation for large language models*. arXiv:2311.07911. IFEval: verifiable instruction constraints ('exactly three bullets'); also an RLVR reward.
- **Zheng et al. (2023).** *Judging LLM-as-a-judge with MT-Bench and Chatbot Arena*. arXiv:2306.05685. LLM judges, their biases (position, verbosity, self-enhancement), and MT-Bench.

- **Dubois et al. (2024).** *Length-controlled AlpacaEval: a simple way to debias automatic evaluators*. arXiv:2404.04475. Why length control matters for judge-based evaluations of preference-tuned models.
- **Li et al. (2024).** *From crowdsourced data to high-quality benchmarks: Arena-Hard and BenchBuilder pipeline*. arXiv:2406.11939. Hard prompts mined from Chatbot Arena with high separability between models.
- **Jain et al. (2024).** *LiveCodeBench: holistic and contamination free evaluation of large language models for code*. arXiv:2403.07974. Continuously refreshed coding problems to defeat contamination.
- **Rein et al. (2023).** *GPQA: a graduate-level Google-proof Q&A benchmark*. arXiv:2311.12022. Expert-written science questions; GPQA Diamond is the reasoning-model standard.
- **Hendrycks et al. (2020).** *Measuring massive multitask language understanding*. arXiv:2009.03300. MMLU: the knowledge benchmark to re-run after every stage, to see what was forgotten.

Index of terms

†, 52
τ-bench, 43

ABF, 34
ablation, 20
accuracy reward, 28
advantage, 16
agent, 44
aha moment, 28
AIME, 28
alignment, 4
alignment tax, 14
Alpaca, 10
AlpacaEval 2 (LC), 48
answer extraction, 23
APIGen, 43
Arena-Hard, 48
assistant, 5
asynchronous RL, 25
attack success rate (ASR), 39
attention temperature, 34
n, 31
Axolotl / LLaMA-Factory, 50

base model, 4
baseline, 20
batch size, 50
BFCL, 43
borderline prompt, 38
Bradley–Terry model, 6
budget forcing, 29

catastrophic forgetting, 9
chain of thought (CoT), 26
chat template, 9
chosen / rejected, 15
chunked prefill, 33
clip-higher, 25
clipping, 16
closed-form optimum, 18
cold start, 28
comparison, 5
confidence interval, 31
consolidation, 47
constitution, 38
Constitutional AI (CAI), 38
constrained fine-tuning, 41
contamination, 48
context distillation, 38
context extension, 32
context window, 32
coverage, 23
credit assignment, 22
critique and revision, 38
curation, 10

DAPO, 25
decoding settings, 49
DeepSeek-R1, 27
DeepSeek-V3-Base, 28
DeepSeekMath, 22
deliberative alignment, 39
demonstration, 5
dense reward, 47
derived, 52
distillation (SFT sense), 10
DPO, 18
Dr. GRPO, 25
dynamic sampling, 25

effective context length, 35
entropy bonus, 31
entropy collapse, 25
environment, 44
epoch, 9
expert iteration, 12
exploration, 30
exposure bias, 9
extrapolation, 33

false refusal rate, 37
fine-tuning attack, 40
FlashAttention, 33
format reward, 23
freeze the suite, 50
FSDP, 25
full fine-tuning, 9
function calling, 42

gate, 35
GCG, 39
generalized advantage estimation (GAE), 17

generative reward model, 15
GKD, 47
gold answer, 23
gold reward, 15
Goodhart's law, 15
GPQA, 48
group, 22
group-relative advantage, 22
GRPO, 20

HarmBench, 39
held-out judge, 49
HH-RLHF, 14
HHH, 37

IcePop, 25
IFEval, 23
illustrative, 52
implicit reward, 18
importance ratio, 16
input guard / output guard, 39
InstructGPT, 13
instruction tuning, 10
IPO, 18
iterative DPO, 19

jailbreak, 39
judge, 12

Kimi K2, 44
KL coefficient, 17
KL divergence, 15
KL estimator, 22
KTO, 18

labeler, 13
language mixing, 28
language-consistency reward, 28
latent ability, 27
learning rate, 50
length-controlled win rate, 48
LIMA, 10
LiveCodeBench, 48
Llama 3, 20
Llama Guard, 39
logistic function, 15
LongRoPE, 34
LoRA, 9
loss mask, 9

lost in the middle, 35

majority voting, 28
many-shot jailbreaking, 39
mid-training, 5
MMLU, 48
MT-Bench, 48
multi-teacher OPD, 47
MuonClip, 45

needle in a haystack (NIAH), 35
negative example, 43
NeMo Gym, 44
non-thinking mode, 30
NTK-aware scaling, 34

o1, 27
objective, 6
off-policy, 18
OLMo 3, 20
on-distribution, 11
on-policy, 17
on-policy distillation (OPD), 47
open-weight model, 4
OpenRLHF, 44
OpenThoughts3, 50
ORPO, 19
outcome supervision, 27
overlong penalty, 25
overoptimization, 15
overthinking, 31

packing, 9
partition function, 18
k, 30
pass^k, 43
policy, 6
policy gradient, 6
position encoding, 32
position interpolation (PI), 34
post-training, 4
PPO, 14
preference optimization, 5
prefix attack, 40
pretraining, 5
PRM800K, 26
process supervision, 26
proxy reward, 15

QLoRA, 9

R1-Zero, 28
ranking, 14
reasoning trace, 29
recipe, 4
recovery data, 41
red teaming, 38
reference model, 17
regression, 51
REINFORCE, 20
reinforcement learning (RL), 5
rejection sampling, 11
release notes, 46
reverse KL, 47
reward, 5
reward hacking, 23
reward model (RM), 6
RL-Zero, 45
RLAIF, 15
RLHF, 13
RLOO, 20
RLVR, 23
RMU, 41
rollout, 17
rollout engine, 25
RoPE, 32
RoPE base, 34
rotation frequency, 34
round, 12
rubric, 15
rubric reward, 23
RULER, 35
runtime, 43

s1, 29
safety policy / spec, 39
safety reward model, 38
sampling budget, 22
sandbox, 23
scalar head, 13
scorer, 11
self-critique, 23
Self-Instruct, 10
sequence parallelism, 33
sequence-level reward, 22
shallow safety alignment, 40
sharpening, 30
short-context regression, 35
SimPO, 18
slime, 44
special token, 9
specialist teacher, 46
stage, 51
staged RL, 46
staleness, 25
STaR, 12
strong-to-weak distillation, 29, 47
supervised fine-tuning (SFT), 5
SWE-bench, 44
synthetic data, 6
synthetic long-context data, 36

tamper-resistant training (TAR), 41
task template, 10
teacher / student, 29
test-time scaling, 26
thinking budget, 29
thinking mode fusion, 29
thinking tokens, 26
tokenizer sharing, 47
tool schema, 42
tool turn, 42
ToolBench, 43
train/inference mismatch, 25
trainer, 25
trajectory, 44
transfer attack, 39
TRL, 50
turn-level advantage, 44
Tülu 3, 10

UltraFeedback, 50
unlearning, 41
up-sampling, 33
uplift evaluation, 41

value model, 17
verifier, 6
veRL / HybridFlow, 44
violation rate, 37

wavelength, 34
weight sync, 25
WMDP, 41

XSTest, 37

YaRN, 34